Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

SZOFTVERFEJLESZTŐ ÉS -TESZTELŐ

Vizsgaremek

PLANOVA

Készítette: Gercsák Tamás
Hallgató László
Rab Mihály József

2026.

SZOFTVER CÉLJA _____ *3*

FELHASZNÁLÓI DOKUMENTÁCIÓ _____ *4*

ADATBÁZIS _____ *10*

FRONTEND _____ *18*

BACKEND _____ *37*

TESZT DOKUMENTÁCIÓ _____ *57*

PLANOVA – SZOFTVER DOKUMENTÁCIÓ

A “*”-gal megjelölt elemek, sorok fejlesztés alatt állnak vagy jövőbeli terveket tükröznek.

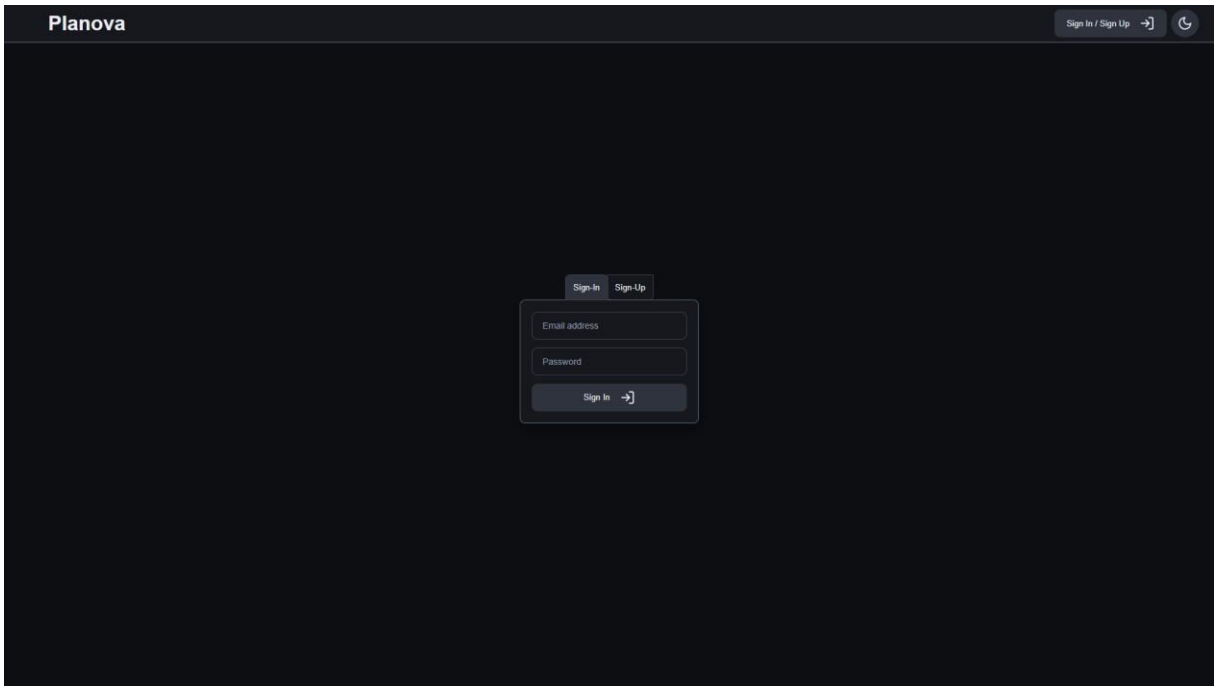
SZOFTVER CÉLJA

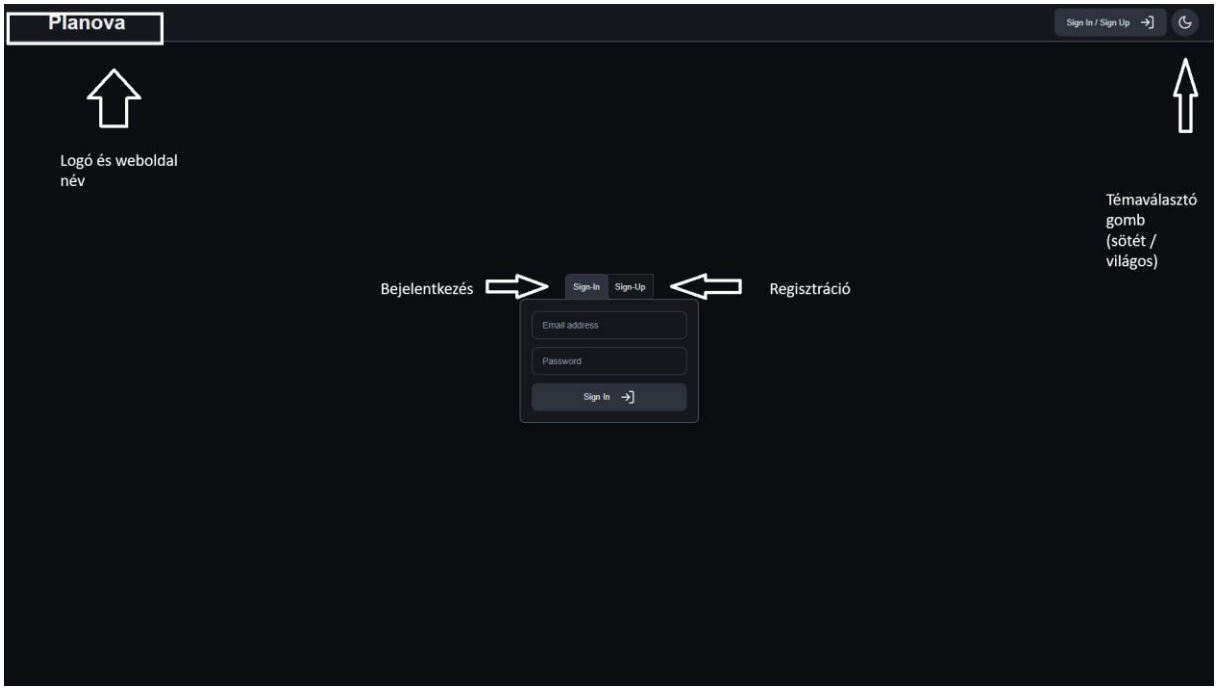
A Planova egy felhasználóközpontú naptár- és produktivitás-alkalmazás, amely lehetővé teszi a felhasználók számára naptárak kezelését, események és feladatok ütemezését, *(szokások követését, barátok kezelését, valamint naptárak megosztását másokkal). Az alkalmazás reszponzív webes felületen érhető el, mely lehetővé teszi hogy számítógépről illetve mobil eszközökről is egyaránt használható.

A szoftver célja, hogy egyetlen platformon egyesítse a személyes időkezelés és produktivitás-követés legfontosabb eszközeit, miközben biztosítja a valós idejű együttműködést és a több eszközön átívelő szinkronizációt.

FELHASZNÁLÓI DOKUMENTÁCIÓ

Verzió: 1.0



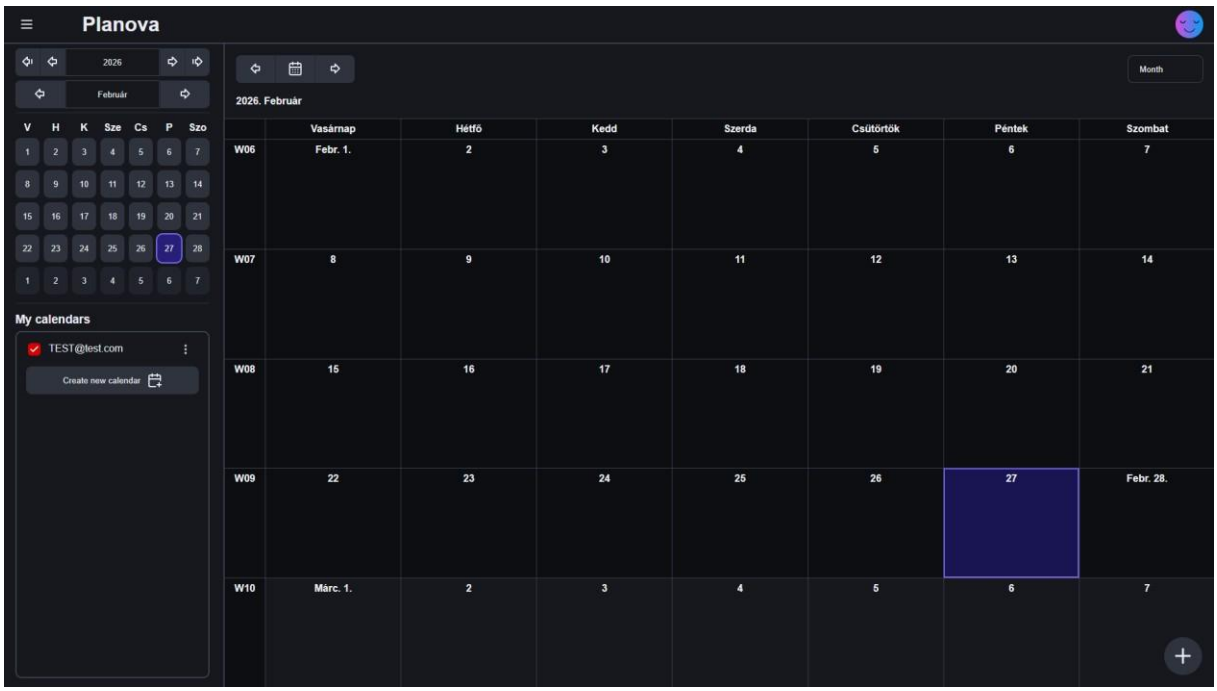


A weboldal megnyitásakor a bejelentkező felület fogadja a felhasználót.

- **Regisztráció** fül: új felhasználói fiók létrehozása

- **Bejelentkezés** fül: meglévő fiókba történő belépés

Sikeres bejelentkezést követően a rendszer a **naptárnézetre** irányítja a felhasználót.

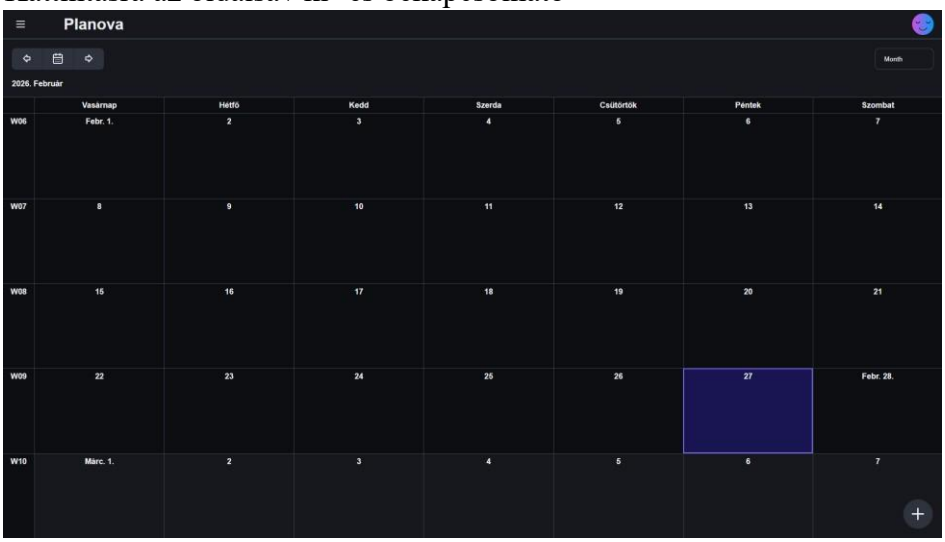


Navigációs sáv

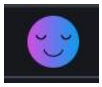
A felső navigációs sáv az alábbi elemeket tartalmazza:

Bal oldal

- **Hamburger menü ikon**
  - Az oldalsáv megjelenítéséért és elrejtéséért felel
  - Kattintásra az oldalsáv ki- és bekapcsolható

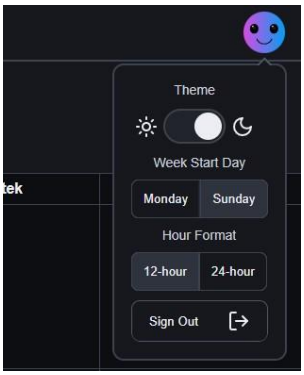


Jobb oldal

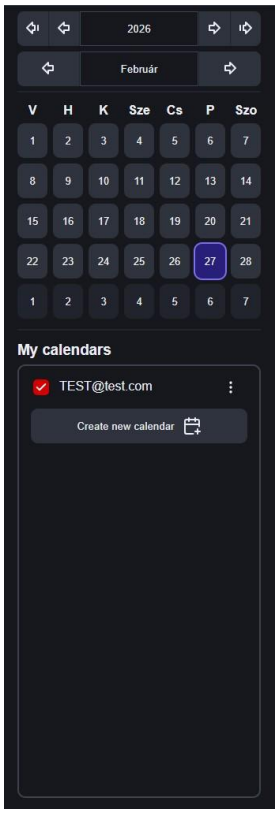
- **Gyorsbeállítások ikon** 

Itt az alábbi beállítások módosíthatók:

- Téma (megjelenés)
- A hét kezdőnapja
- Óraformátum (12 / 24 órás)



Oldalsáv



Az oldalsáv a gyors navigációt segíti.

Funkciói:

- Dátum gyors módosítása
- A fiókhoz csatolt naptárak megjelenítése a **„My Calendars”** szekcióban

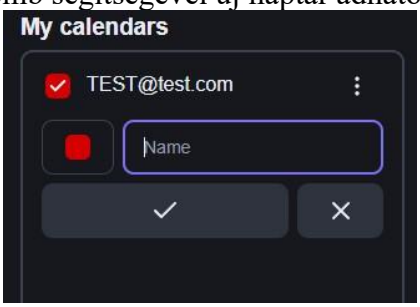
Új fiók létrehozásakor a rendszer automatikusan generál egy alapértelmezett naptárat (pl.: felhasznalo@email.com).

Új naptár létrehozása

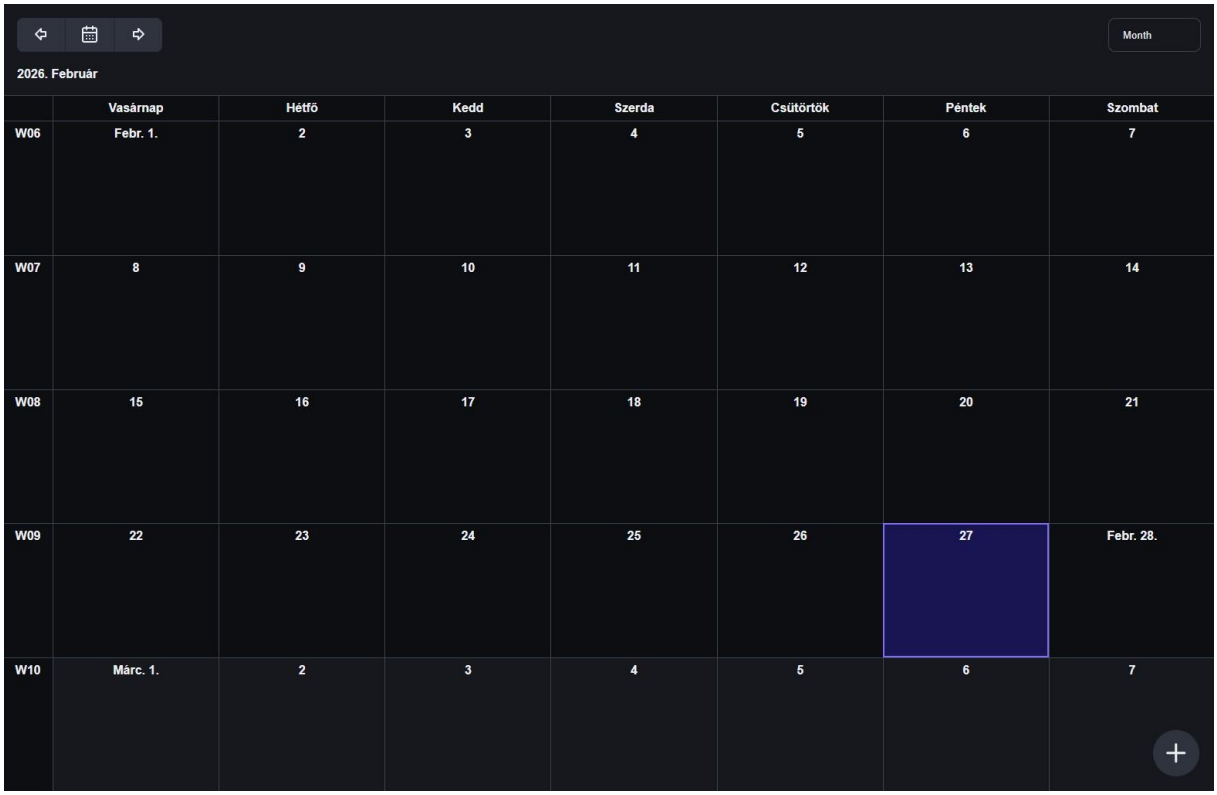
A **„Create New Calendar”** gomb segítségével új naptár adható hozzá.

Beállítható:

- A naptár neve
- A naptár színe



Naptárnézet



Bal felső sarok gombjai:

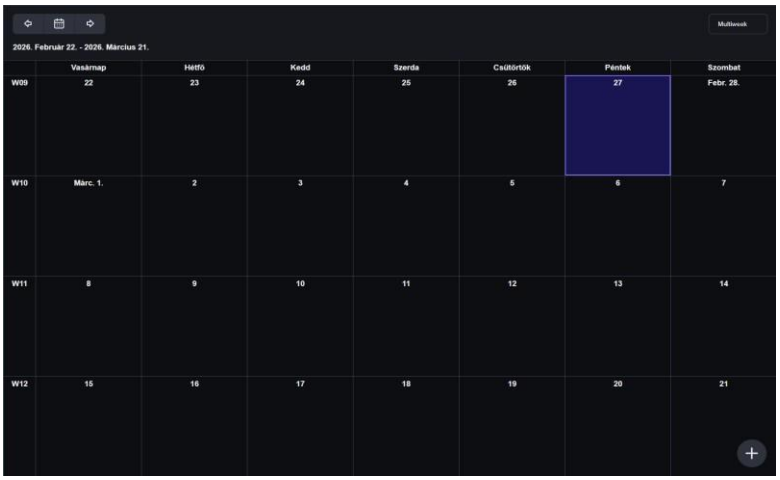
- **Balra / Jobbra nyíl** o Egy hónappal vissza vagy előre lép
- **Naptár ikon** o Visszaugrás az aktuális dátumhoz

Jobb felső sarok

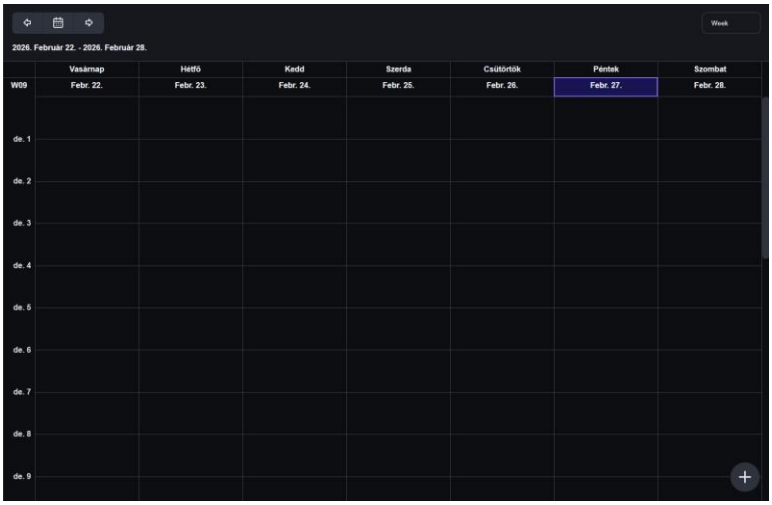
- **„Month” gomb**
  - o A nézet módosítására szolgál

Elérhető nézetek:

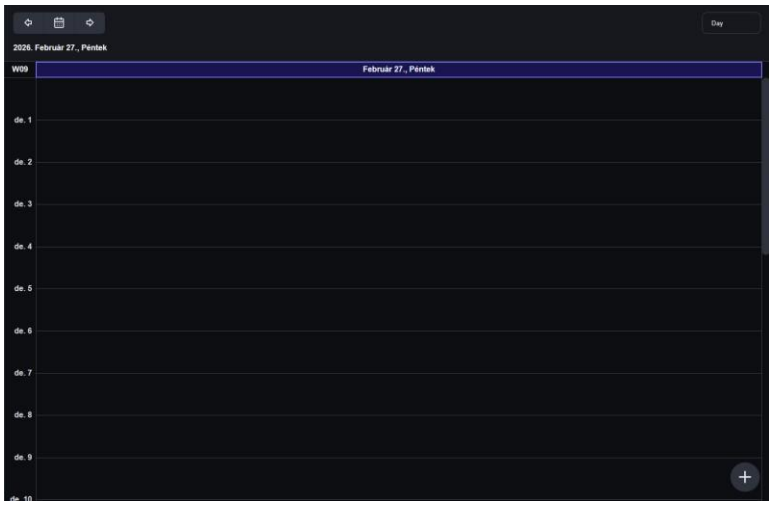
- Havi nézet
- Többhetes nézet (4 hét)



- Heti nézet



- Napi nézet



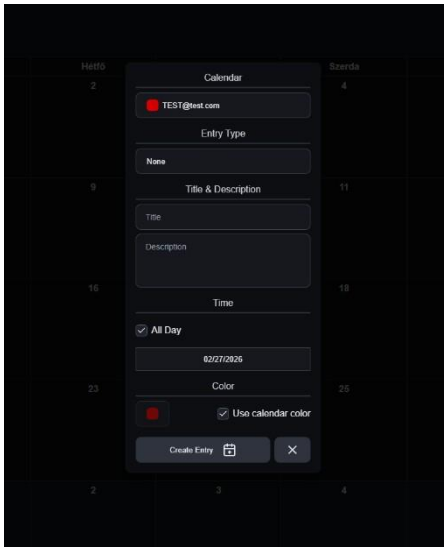
További jelölések

- A bal oldali oszlopban láthatók az adott év hetei (pl.: W06, W07)
- A kék színnel jelölt cella az aktuális dátumot mutatja

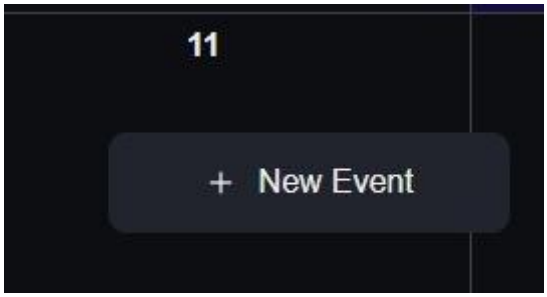
A jobb alsó sarokban található gomb segítségével új esemény hozható létre.

Az esemény létrehozásakor megadható:

- Melyik naptárhoz tartozzon
- Az esemény neve • Leírás (opcionális) • Időtartam:
    - Egész napos
    - Időintervallumhoz kötött
- Szín
    - Alapértelmezés szerint a kiválasztott naptár színét örökli



- Ugyan ez a funkció elérhető számítógépen a jobb egérgomb lenyomásával



ADATBÁZIS

Áttekintés

Adatbázis neve: Vizsgaremek

RDBMS (Relational Database Management System) típusa: PostgreSQL 18

Verzió: v1.1.0

Utolsó frissítés: 2025.12.15

Tábla: Users

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A felhasználó egyedi azonosítója
email	text	required, unique	A felhasználó email címe, amit belépéskor és regisztráláskor használhat
password_hash	text	required	A felhasználó hash-elt jelszava
refresh_token	text	nullable	Hosszú életű hitelesítési kulcs, amivel új access token igényelhető a felhasználó bejelentkezése nélkül.
refresh_token_expiry_time	timestamp	nullable	A refresh token lejárati dátuma
created_at	timestamp	default: Now()	A felhasználó létrehozásának ideje

Indexek

- id (clustered)
- email (non-clustered)

ON DELETE CASCADE

Felhasználó törlése esetén az összes hozzá tartozó adat törlődik, beleértve a hozzá tartozó profilt, a barátokat, a naptárakat, eventeket, taskokat. Csak azok az adatok törlődnek, amiket a felhasználó maga hozott létre. Megosztott naptár esetén a naptár törlésére vonatkozó szabályok lépnek érvénybe.

Tábla: Profiles

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A felhasználó profiljának egyedi azonosítója
username	text	length: 3-20, required, unique	
avatar	text	required, default: placeholder	A felhasználó által feltöltött kép

is_private	boolean	default: true	A felhasználó eldöntheti, hogy az adatai nyilvánosak legyenek-e
first_name	text	nullable, max. length: 128	Keresztnév
last_name	text	nullable, max. length: 128	Családnév
birth_date	timestamp	default: Now()	Születési dátum
user_id	uuid	FK	Hivatkozás a Users táblára

Indexek

- id(clustered),
- user_id(non-clustered),
- username(non-clustered)

ON DELETE

Profil törlése esetén a felhasználó maga is törlődik, így ugyanazok a szabályok érvényesek

Tábla: Calendars

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A naptár egyedi azonosítója
name	text	length: 3-20, required	A naptár neve
profile_id	uuid	FK	Hivatkozás a Profiles táblára
color	int	range: 1-12, default: 1	A naptár megjelenítési színe

Indexek

- id(clustered)
- profile_id(non-clustered)

ON DELETE CASCADE

Alapértelmezetten törlődik a naptár és a hozzá tartozó események és feladatok. Megosztott naptár esetén a naptár automatikusan öröklődik az első személyhez, akivel a naptár meg van osztva.

Tábla: Shared_Calendars

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
profile_id	uuid	PK	Hivatkozás a Profiles táblára
calendar_id	uuid	PK	Hivatkozás a Calendars táblára
role	enum	default: Viewer	A naptárhoz hozzáfért személy szerepköre

Kapcsoló tábla, a profile_id és a calendar_id együttesen határozza meg a táblát - composite key

ON DELETE

A felhasználóval megosztott naptár számára már nem elérhető

Role enum

A felhasználó szerepkörének meghatározása megosztott naptár esetén. Lehetséges értékek:

- Viewer (0),
- Editor (1)

Tábla: Calendar_entries

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A feladat egyedi azonosítója
entry_category	enum	required	A bejegyzés kategóriája
name	text	length: 3-32, required	A feladat megnevezése
description	text	nullable, max. length: 1024	A bejegyzés rövid leírása
start_date	timestamp	required	A bejegyzés kezdő dátumának megadása.
end_date	timestamp	nullable	A bejegyzés befejezési idejének megadása.
location	geography	nullable	A bejegyzéshez tartozó helyszín
notification_time	timestamp	nullable	Ha a felhasználó szeretné, kérhet értesítést. Ennek az idejét tároljuk itt
color	int	range: 1-12	A bejegyzés megjelenítési színe
is_completed	boolean	nullable	A feladat teljesülését jelző érték.
created_by	uuid	FK	Hivatkozás a Profiles táblára.
calendar_id	uuid	FK	Hivatkozás a Calendars táblára.

Indexek

- id(clustered)
- calendar_id(non-clustered)
- created_by(non-clustered)

Entry_Category enum

Az esemény kategóriájának megjelölése. Lehetséges értékek:

- Event (0),
- Task (1),
- BirthDay (2),
- Anniversary (3)

ON DELETE CASCADE

Törlődik a bejegyzés és ha van, az Event_Contributors táblában lévő összes rekord ezzel az eseménnyel

Tábla: Event_Contributors

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
profile_id	uuid	PK	Hivatkozás a Profils táblára
event_id	uuid	PK	Hivatkozás az Events táblára
status	enum	required	Az esemény elfogadása

Kapcsoló tábla, a profile_id és az event_id együttesen határozza meg a táblát - composite key. Ha nincs megosztott naptár egy személlyel, de szeretnénk meghívni valakit egy eseményre, ebben a táblában tudjuk eltárolni a hozzáadott embereket.

ON DELETE

A felhasználóval megosztott esemény számára már nem elérhető

Tábla: Friends

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
user1_profile_id	uuid	PK	Hivatkozás a Profiles táblára
user2_profile_id	uuid	PK	Hivatkozás a Profiles táblára
status	enum	alapértelmezett: Pending	A barátkérés állapota
created_at	timestamp	alapértelmezett: Now()	A barátkérés elküldésének ideje
updated_at	timestamp	lehet üres	A státusz változásának ideje

Kapcsoló tábla, ahol a barát kapcsolatokat lehet tárolni. A kettő profile_id együttesen határozza meg a táblát - composite key

ON DELETE

A felhasználók közötti kapcsolat törlődik.

Status Enum

A barátkérés állapotát jelzi. Lehetséges értékek:

- Pending (0),
- Yes (1),
- No (2)

Tábla: Habits

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A szokás egyedi azonosítója
name	text	hossz: 3-64, kötelező	A szokás megnevezése
description	text	lehet üres, max. hossz: 1024	A szokás leírása
habit_category	enum	kötelező, alapértelmezett: None	A szokás kategóriája
unit	enum	kötelező, alapértelmezett: None	A szokás teljesítését jelző mértékegység
goal	float	lehet üres	A egy szokás egy alkalomhoz tartozó cél mennyisége. (Mennyit?)
color	int	tartomány: 1-24	Az esemény megjelenítési színe
days	int	bitmask, required	A szokás napjait jelző érték.
profile_id	uuid	FK	Hivatkozás a Profiles táblára.

Indexek

- id(clustered)
- profile_id(non-clustered)

ON DELETE

CASCADE - Törlés esetén a szokás és a hozzátartozó logok is törlődnek

Habit_Category Enum

A szokás kategóriáját jelzi. Lehetséges értékei:

- None (0),
- Exercise (1),
- Reading (2),
- Mindfulness (3)

Unit Enum

A szokás teljesítését jelző mértékegység. Lehetséges értékek:

- None (0),
- Distance (1),
- Quantity (2),
- Time (3)

Tábla: Habit_Logs

Oszlop neve	Adattípus	Megszorítások	Leírás/megjegyzés
id	uuid	PK	A szokás bejegyzésének egyedi azonosítója
habit_id	uuid	FK	Hivatkozás a Habits táblára
date	timestamp	alapértelmezett: Now()	A bejegyzés dátuma
value	float	lehet üres	Az adott dátumon elért szokáshoz tartozó mennyiség jelölése.

A habit logok automatikusan létrehozódnak a meghatározott napokon.

Indexek:

- id(clustered)
- habit_id(non-clustered)

ON DELETE:

A szokáshoz tartozó bejegyzés törlődik.

FRONTEND

A Planova egy többplatformos naptáralkalmazás, amelynek frontend oldala két klienst tartalmaz:

- Web alkalmazás (Next.js + Tamagui)
- Mobil alkalmazás (Expo + React Native + Tamagui) *

Mindkét kliens ugyanazokat a megosztott csomagokat (packages) használja, ami biztosítja a kód újrafelhasználhatóságát és az egységes üzleti logikát. A projekt monorepo struktúrát alkalmaz pnpm workspace és Turborepo segítségével.

Technológiai stack

Közös technológiák

React 19 - UI könyvtár

- TypeScript - Típusbiztos JavaScript
- Tamagui - Univerzális UI keretrendszer (web + native)
- Zustand - Állapotkezelés (state management)
- TanStack React Query - Szerver-oldali állapotkezelés és cache
- Axios - HTTP kérések kezelése
- Zod - Séma validáció (űrlapok)
- React Hook Form - Űrlapkezelés
- SignalR (@microsoft/signalr) - Valós idejű értesítések (WebSocket)

Web alkalmazás (apps/web)

- Next.js (Turbopack) - React keretrendszer SSR/SSG támogatással
- @tamagui/next-plugin - Tamagui integráció Next.js-hez
- @tamagui/next-theme - Téma kezelés (sötét/világos mód)
- react-datepicker - Dátumválasztó komponens
- react-loading-indicators - Betöltési animációk
- Lucide Icons (@tamagui/lucide-icons) - Ikonkészlet

Mobil alkalmazás (apps/mobile) *

- Expo - React Native fejlesztői platform
- Expo Router - Fájl-alapú navigáció
- expo-secure-store - Biztonságos token tárolás
- React Navigation - Navigációs könyvtár (tab-ok, stack-ek)
- react-native-reanimated - Animációk
- react-native-gesture-handler - Gesztuskezelés
- react-native-screens - Natív képernyők optimalizálása

Fejlesztői eszközök

- pnpm - Csomagkezelő (v10.28.0)
- Turborepo - Monorepo build rendszer
- ESLint - Kódminőség-ellenőrzés
- openapi-typescript - API típusgenerálás Swagger-ből
- Docker / Docker Compose - Konténerizáció

Projekt struktúra

Monorepo felépítés

```
Vizsgaremek/
├── apps/
│   ├── web/ # Next.js webalkalmazás
│   ├── mobile/ # Expo mobil alkalmazás *
│   └── backend/ # .NET backend (C#)
├── packages/
│   ├── api/ # API hívások (axios)
│   ├── config/ # Tamagui konfiguráció és témák
│   ├── features/ # Megosztott feature komponensek
│   ├── hooks/ # Zustand store-ok és React Query hook-ok
│   ├── types/ # TypeScript típusdefiníciók + Swagger típusok
│   ├── ui/ # Újrafelhasználható UI komponensek
│   └── utils/ # Segédfüggvények és naptárlogika
├── scripts/ # Fejlesztői scriptek
├── docker-compose.yml # Docker Compose konfiguráció
├── pnpm-workspace.yaml # pnpm workspace konfiguráció
├── turbo.json # Turborepo konfiguráció
└── package.json # Gyökér csomag
```

Web alkalmazás struktúra (apps/web)

```
apps/web/
├── next.config.ts # Next.js + Tamagui konfiguráció
├── Dockerfile # Docker build a webalkalmazáshoz
```

```
├── package.json # Függőségek
├── public/
│   └── tamagui.css # Tamagui CSS kimenet
└── src/
    ├── app/
    │   ├── layout.tsx # Gyökér layout (HTML, Providers)
    │   ├── page.tsx # Főoldal (átirányít /sign-in-re)
    │   ├── globals.css # Globális CSS stílusok
    │   ├── sign-in/
    │   │   └── page.tsx # Bejelentkezési oldal
    │   ├── testpage/
    │   │   └── page.tsx # Tesztoldal
    │   └── (dashboard)/ # Védett route csoport
    │       ├── layout.tsx # Dashboard layout (SignalR + védelem)
    │       ├── calendar/
    │       │   └── page.tsx # Naptár főoldal
    │       └── profile/
    │           └── page.tsx # Profil oldal (fejlesztés alatt)
    └── components/
        ├── Providers.tsx # Alkalmazás provider-ek (téma, query, auth)
        ├── ProtectedRoute.tsx # Hozzáférés-védelmi wrapper
        ├── FullscreenView.tsx # Teljes képernyős elrendezés komponens
        ├── Navbar/
        │   ├── Navbar.tsx # Navigációs sáv (publikus + privát)
        │   └── ThemeToggle.tsx # Sötét/világos mód váltó
        └── CalendarPage/
            ├── Sidebar.tsx # Oldalsáv (naptárkezelés)
            ├── CalendarHeader.tsx # Naptár fejléc (navigáció, nézetek)
            ├── CalendarEntryDetail.tsx # Bejegyzés részletei
            ├── CalendarEntryHourView.tsx # Óra nézet bejegyzésekhez
```

```
├── ColorSelect.tsx # Szín választó
├── ContextMenu.tsx # Jobb-klikk kontextus menü
├── CreateCalendarForm.tsx # Naptár létrehozó űrlap
├── CreateCalendarEntryForm.tsx # Bejegyzés létrehozó űrlap
├── UpdateCalendarForm.tsx # Naptár szerkesztő űrlap
└── CustomDialog.tsx # Egyedi dialógus ablak
```

Mobil alkalmazás struktúra (apps/mobile) *

```
apps/mobile/
├── app.json # Expo konfiguráció
├── babel.config.js # Babel konfiguráció
├── metro.config.js # Metro bundler konfiguráció
├── package.json # Függőségek
├── app/
│ ├── _layout.tsx # Gyökér layout (auth, providerek)
│ ├── index.tsx # Belépési pont (auth-alapú átirányítás)
│ ├── (guest)/ # Vendég útvonalak
│ │ ├── _layout.tsx # Tab navigáció (Home, Sign-In)
│ │ ├── home.tsx # Kezdőlap
│ │ └── sign-in.tsx # Bejelentkezés
│ └── (protected)/ # Védett útvonalak
│ ├── _layout.tsx # Tab navigáció (Calendar)
│ └── calendar.tsx # Naptár oldal
└── android/ # Android natív projekt
└──
```

Megosztott csomagok (packages/)

Minden csomag workspace hivatkozással érhető el: @repo/<csomag_név>

packages/api/ → @repo/api
packages/config/ → @repo/config
packages/features/ → @repo/features
packages/hooks/ → @repo/hooks
packages/types/ → @repo/types

packages/ui/ → @repo/ui packages/utils/
→ @repo/utils

Megosztott csomagok részletes leírása

@repo/api – API kommunikációs réteg

Az API csomag felelős a backend-del való minden HTTP kommunikációért.

Fájlok:

- api.ts - Axios instance konfigurálása, interceptorok
- auth.ts - Autentikációs végpontok (signUp, signIn, signOut, verify, refresh)
- calendar.ts - Naptár CRUD műveletek (getCalendar, createCalendar, updateCalendar, deleteCalendar)
- calendarEntry.ts - Naptárbejegyzés CRUD (getCalendarEntry, createCalendarEntry, updateCalendarEntry, deleteCalendarEntry)
- profile.ts - Profil műveletek (getProfile, updateProfile)
- notification.ts - Barát kérés küldése (sendFriendRequest)
- nativeTokenStorage.ts - Token tárolás kezelése (natív vs. web)

Axios instance jellemzők:

- Alap URL: http://localhost:8080
- Web: cookie-alapú hitelesítés (withCredentials: true)
- Natív: Bearer token hitelesítés (expo-secure-store)
- Automatikus token frissítés interceptorral (401-es hiba esetén)
- Kérés sorba állítás token frissítés közben (failedQueue) Token kezelés:

- Web: böngésző cookie-kat használ (a szerver állítja be)
- Natív: expo-secure-store-ban tárolt accessToken és refreshToken
- Az initializeNativeTokenStorage() függvénnyel injektálható a SecureStore

@repo/hooks – Állapotkezelés (Zustand store-ok és React Query hook-ok)

Zustand store-ok:

useAuthStore

Állapot: isAuthorized, isLoading

Műveletek: verifyAuth() – token érvényesítés és automatikus refresh
Folyamat: verify() → hiba esetén refresh() → hiba esetén
isAuthorized=false

useCalendarStore

Állapot: currentDate, selectedDate, viewType, checkedCalendarIds,
tabletView, mobileView, hideSidebar

Nézet típusok: "month" | "multiweek" | "week" | "day"

Navigáció: decMonth/incMonth, decWeek/incWeek, decDay/incDay, decYear/incYear, resetToToday

Reszponzivitás: initResizeListener() – ablakméret változás figyelése

(768px=mobil, 1024px=tablet)

useProfileStore

Állapot: locale (Intl.Locale), weekStartsOn, hour12

Perzisztencia: localStorage-ba menti a beállításokat

Naptár kezdőnapja: "monday" | "sunday"
Óra formátum: 12 órás vagy 24 órás

useContextMenuStore

Jobb-klikk kontextus menü kezelése

Állapot: position, display, contextMenuOpen, fieldType, date
Típusok: "cell" (üres cella) | "schedule" (meglévő bejegyzés)

useDialogStore

Felugró dialógus ablakok kezelése

Állapot: isDialogOpen, title, description, content (React node)

useNavbarStore

Navigációs sáv állapota (beállítások panel nyitva/zárva)

useNotificationStore

SignalR kapcsolat kezelése (valós idejű értesítések)

Állapot: connection, notifications, isConnected

SignalR Hub: /notifications végpont

Automatikus újracsatlakozás (.withAutomaticReconnect())
Web: cookie-kat használ | Natív: access token-t küld

React Query hook-ok:

useCalendars

Lekéri az összes naptárat (GET api/Calendar)

Cache: 5 perc staleTime

Query kulcs: ["myCalendars"]

useCalendarEntries

Lekéri a naptárak bejegyzéseit (párhuzamos kérések minden naptárnak)

Feltétel: myCalendarsQuery.data elérhetősége
Cache: 5 perc staleTime

Query kulcs: ["calendarEntries", calendarIds]

useProfile

Profil adatok lekérése

@repo/features – Megosztott feature komponensek

Hitelesítés:

- AuthCard: Toggle-özhető bejelentkezés/regisztráció kártya
- SignInForm: Bejelentkezési űrlap (email + jelszó, Zod validáció)
- SignUpForm: Regisztrációs űrlap (email + jelszó erős validációval)

Jelszó követelmények (SignUpForm):

- Minimum 8, maximum 128 karakter
- Legalább 1 kisbetű
- Legalább 1 nagybetű
- Legalább 1 szám
- Legalább 1 speciális karakter

Naptár:

- Calendar: Fő naptár komponens (havi, többhetes, heti, napi nézet)
- CalendarCell: Egyedi naptár cella komponens
- CalendarEntry: Naptárbejegyzés megjelenítése

@repo/ui – Újrafelhasználható UI komponensek

- StyledButton - Egységes stílusú gomb
- StyledInput - Egységes stílusú szövegbeviteli mező
- StyledTextArea - Egységes stílusú szövegterület
- SelectElement - Egyedi legördülő menü
- ColorIcon - Szín ikon megjelenítés
- ToggleGroupItemText - Toggle csoport szöveges elem

A Tamagui keretrendszer biztosítja, hogy ezek a komponensek mindkét platformon (web és natív) azonos kinézettel és viselkedéssel működnek.

@repo/types – Típusdefiníciók

- calendar.ts - Naptár típusok: CalendarViewType, CalendarCellProps, WeekStartDay, MonthLabel, WeekDayLabel stb.
- swagger.ts - Automatikusan generált típusok a backend Swagger/OpenAPI specifikációjából (openapi-typescript)

Nézet típusok (CalendarViewType):

"month" – havi nézet

"multiweek" – többhetes nézet (4 hét)

"week" – heti nézet
"day" – napi nézet

Naptár cella (CalendarCellProps):

- date: Date – a cella dátuma
- inCurrentMonth: boolean – az adott hónapba tartozik-e

@repo/utils – Segédfüggvények

Naptár logika:

- calendarGridFactory.ts - Naptár rács generálás különböző nézetekhez generateMonthCells() – 35/42 cella a havi nézethez generateMultiweekCells() – 28 cella (4 hét) a többhetes nézethez generateWeekCells() – 7 cella a heti nézethez generateDayCell() – 1 cella a napi nézethez generateGrid() – fő hívó függvény, hét számok szerinti csoportosítás
- dateMethods.ts - Dátum segédfüggvények
- Month.ts - Hónap osztály
- Week.ts - Hét osztály
- calendarEntryUtils.ts - Bejegyzés pozicionálási logika Szín

  segédlet:

- colors.ts - getContrastFromHSLA(): HSLA szín alapján visszaadja,

  hogy fekete vagy fehér szövegszín szükséges a megfelelő

  kontraszthoz (YIQ képlet) Platform detektálás:

- isNative.ts - isNative(): meghatározza, hogy React Native környezetben fut-e az alkalmazás

@repo/config – Tamagui konfiguráció

- tamagui.config.ts - Tamagui teljes konfigurációja (a defaultConfig v4-re épül)
- fonts.ts - Betűtípus beállítások
- themes.ts - Téma definíciók:
  Alap paletta: 12 szín sötét és világos módhoz (HSLA formátum)

  Accent paletta: Kiemelő színek (kék árnyalatok)

  Gyerek témák: warning (sárga), error (piros), success (zöld), info (kék), calendarColors (12 naptár szín)
  Árnyékok: lightShadows, darkShadows (6 szint)

Alkalmazás működése

Indítás és hitelesítés

Betöltési folyamat:

1. Az alkalmazás elindul, a Providers komponens betöltődik
2. Az useAuthStore.verifyAuth() meghívódik
3. Először verify() API hívás történik (token érvényesítés)
4. Ha sikertelen, refresh() hívás (token frissítés)
5. Ha a frissítés is sikertelen: isAuthorized = false

6. A betöltési állapot alatt egy animált OrbitProgress jelenik meg

Web navigáció:

- / → átirányítás /sign-in-re
- /sign-in → bejelentkezés/regisztráció oldal (ha autorizált → /calendar)
- /calendar → naptár oldal (védett)
- /profile → profil oldal (fejlesztés alatt)

Mobil navigáció: *

- index.tsx → auth ellenőrzés → (guest)/sign-in VAGY (protected)/calendar
- (guest) csoport: Home tab + Sign-In tab
- (protected) csoport: Calendar tab

Hitelesítési folyamat

Bejelentkezés (SignInForm):

1. Felhasználó megadja az email címet és jelszót
2. Zod validáció ellenőrzi a mező kitöltöttségét
3. signIn() API hívás (POST api/auth/sign-in)
4. Webre: a szerver cookie-kat állít be
5. Nativra: accessToken + refreshToken tárolódik SecureStore-ban
6. Sikeres bejelentkezés után: átirányítás a /calendar oldalra

Regisztráció (SignUpForm):

1. Email + jelszó validáció (erős jelszó követelmény)
2. signUp() API hívás (POST api/auth/sign-up)
3. Automatikus bejelentkezés a signIn() hívással
4. Átirányítás a /calendar oldalra

Kijelentkezés:

1. Token-ek törlése (SecureStore vagy cookie)
2. signOut API hívás (GET api/auth/sign-out)
3. Átirányítás a /sign-in oldalra (window.location.href)

Naptár nézetek

A naptár négy különböző nézetet támogat:

Havi nézet ("month"):

- 5 vagy 6 hetes rács (35 vagy 42 cella) - Hét számok az oldalsávban
- Előző/következő havi napok halványítva - Görgős navigáció (mousewheel)

Többhetes nézet ("multiweek"):

- 4 hetes rács (28 cella)
- Az aktuális hét a kiválasztott dátum alapján

Heti nézet ("week"):

- 7 nap, óra-alapú rács (0-23 óra)
- Oldalra görgethető tartalom

Napi nézet ("day"):

- 1 nap, 24 órás rács
- Részletes időbeosztás megjelenítés Navigáció:

- Előző/következő gombokkal (nyílak)
- Egérgörgővel (havi nézetben)
- "Ma" gombbal (resetToToday)
- Nézet típus váltó (legördülő menü)

Oldalsáv (Sidebar)

Az oldalsáv tartalmazza:

- DatePicker: mini naptár a dátumválasztáshoz
- Naptárlista: a felhasználó naptárai checkbox-okkal
- Naptár szín jelzése
- Naptár szerkesztése (név, szín módosítás)
- Naptár törlése
- Új naptár létrehozása (CalendarPlus ikon + CreateCalendarForm)
- Reszponzív: tablet nézetben elrejthető (hamburger menü)

Naptárbejegyzések

Bejegyzés létrehozása:

1. Jobb-klikk egy cellára → kontextus menü → "New Event"
2. Vagy a napi/heti nézetben "+" gomb
3. CreateCalendarEntryForm megnyílik dialógusban
4. Mezők: naptár kiválasztás, kategória, név, leírás, egész napos, kezdő dátum, befejező dátum, szín 5. Zod validáció után API hívás

Bejegyzés kategóriák (EntryCategory):

- None (Nincs)
- Task (Feladat)
- Event (Esemény)
- BirthDay (Születésnap)
- Anniversary (Évforduló)

Naptárbejegyzés színek:

12 előre definiált szín áll rendelkezésre (color1-color12), amelyeket a calendarColors téma definiál.

Kontextus menü

A jobb-klikk kontextus menü a ContextMenuStore-on keresztül működik:

- Cella típus ("cell"): Új bejegyzés létrehozása
- Bejegyzés típus ("schedule"): Bejegyzés szerkesztése/törlése
- Pozíció automatikus igazítása az ablak széleihez
- Bal klikk a menün kívülre bezárja

Dialógus ablakok

A CustomDialog komponens:

- Modal megjelenítés (háttér elsötétítés)
- Testreszabható cím, leírás és tartalom
- Animált be/kimenet
- Maximális magasság korlátozás (90vh)
- Globálisan is használható a useDialogStore-on keresztül

Valós idejű értesítések (SignalR)

- A dashboard layout indítja a SignalR kapcsolatot
- Hub végpont: /notifications
- Automatikus újracsatlakozás
- Web: cookie-alapú hitelesítés | Natív: Bearer token
- ReceiveNotification esemény fogadása
- Értesítések listában tárolódnak (notifications tömb)

Navigációs sáv (Navbar)

Két változat létezik az autorizáció állapotától függően:

Publikus Navbar (nem bejelentkezett felhasználó):

- Planova logó + alkalmazás neve
- "Sign In / Sign Up" gomb
- Téma váltó gomb (kör alakú)

Privát Navbar (bejelentkezett felhasználó):

- Hamburger menü (oldalsáv nyitás/zárás)
- Planova logó + alkalmazás neve
- Avatar (kattintásra beállítások panel) Beállítások panel (Popover):

- Téma váltó (sötét/világos mód switch) • Hét kezdőnapja (hétfő/vasárnap toggle)
- Óra formátum (12 órás/24 órás toggle)
- Kijelentkezés gomb

Reszponzív működés

A webalkalmazás reszponzív tervezésű:

- Desktop (> 1024px): Teljes nézet, oldalsáv látható
- Tablet (768px – 1024px): Oldalsáv alapértelmezetten elrejtve, hamburger menüvel megnyitható
- Mobil (< 768px): Mobil nézet, egyszerűsített elrendezés

Az ablakméret figyelése az useCalendarStore.initResizeListener() függvényen keresztül történik, amely resize event listener-t regisztrál.

Téma rendszer

Az alkalmazás sötét és világos módot támogat.

Szín paletta felépítés (12 szint):

1. Background (háttér)
2. Subtle background (enyhe háttér)
3. Inactive surface (inaktív felület)
4. Default surface (alapértelmezett felület)
5. Hover surface (hover állapot)
6. Active surface (aktív felület)
7. Default border (alapértelmezett szegély)
8. Hover border (hover szegély)
9. Muted fill (tompított kitöltés)

10. Strong fill (erős kitöltés)
11. Secondary text (másodlagos szöveg)
12. Primary text (elsődleges szöveg) Speciális témák:

- accent: Kiemelő szín (Google Blue: #1A73E8)
- warning: Figyelmeztetés (sárga - amber)
- error: Hiba (piros - red)
- success: Siker (zöld - green)
- info: Információ (kék - blue)
- calendarColors: 12 naptár szín

Téma tárolás:

- Web: NextThemeProvider kezeli (automatikus rendszerpreferencia detektálás)
- Natív: useColorScheme() alapján (rendszer téma)

Globális CSS változók:

--navbar-height: 60px (navigációs sáv magassága)

--scrollbar-width: 15px (görgetősáv szélessége)
Betűtípus: Roboto (Google Fonts)

Provider struktúra

Web (Providers.tsx)

Providers (legkülső)

└── NextThemeProvider (téma perzisztencia)

└── InnerProviders

├── auth verify (hitelesítés ellenőrzés indításkor)

├── TamaguiProvider (Tamagui UI keretrendszer)

│ └── QueryClientProvider (React Query cache)

│ └── MainView (kontextus menü + dialógus kezelés)

│ ├── ContextMenu (globális jobb-klikk menü)

│ ├── Navbar (navigációs
sáv) │ └── {children} (oldal
tartalma)

Mobil (_layout.tsx) *

QueryClientProvider (React Query cache)

└── TamaguiProvider (Tamagui UI)

└── ThemeProvider (React Navigation téma)

└── Stack (Expo Router navigáció)

├── (guest) (vendég útvonalak)

└── (protected) (védett útvonalak)

API végpontok (frontend oldal)

A frontend az alábbi backend API végpontokat használja:

Autentikáció:

POST api/auth/sign-up - Regisztráció

POST api/auth/sign-in - Bejelentkezés

GET api/auth/sign-out - Kijelentkezés

GET api/auth/verify - Token érvényesítés
GET api/auth/refresh - Token frissítés

Naptár:

GET api/Calendar - Naptárak lekérdezése

POST api/Calendar - Naptár létrehozása

PUT api/Calendar/{id} - Naptár módosítása

DELETE api/Calendar/{id} - Naptár törlése

Naptárbejegyzés:

GET api/CalendarEntry/{id} - Bejegyzések lekérdezése (szűrhető dátummal)

POST api/CalendarEntry - Bejegyzés létrehozása

PUT api/CalendarEntry/{id} - Bejegyzés módosítása
DELETE api/CalendarEntry/{id} - Bejegyzés törlése

Profil:

GET api/Profile - Profil lekérdezése
PUT api/Profile/{id} - Profil módosítása

Barát kérés:

POST api/friend-request - Barát kérés küldése

SignalR Hub:

/notifications - Valós idejű értesítések (WebSocket)

Típusgenerálás (SWAGGER)

A projekt automatikusan generálja a TypeScript típusokat a backend Swagger specifikációjából:

Parancs: pnpm generate-types

Forrás: http://localhost:5273/swagger/v1/swagger.json
Kimenet: packages/types/src/swagger.ts

Ez biztosítja, hogy a frontend típusai mindig szinkronban legyenek a backend API-val. A generált típusok a components["schemas"] névtéren keresztül érhetők el, például:

- components["schemas"]["SignInRequestDTO"]
- components["schemas"]["GetCalendarDTO"]
- components["schemas"]["CreateCalendarEntryDTO"]
- components["schemas"]["GetProfileDTO"]

- components["schemas"]["EntryCategory"]
- components["schemas"]["TokenResponseDTO"]

Build és deployment

Fejlesztési parancsok

pnpm dev - Interaktív dev script (backend + frontend) pnpm dev:all - Összes service indítása párhuzamosan pnpm web:dev - Csak a webalkalmazás indítása pnpm mobile:dev - Csak a mobil alkalmazás indítása pnpm backend:dev - Csak a backend indítása pnpm generate-types - Swagger típusgenerálás pnpm build - Teljes build (Turborepo) pnpm test - Tesztek futtatása pnpm reinstall - Csomagok újratelepítése pnpm clean:packages - Csomagok törlése pnpm clean:android - Android build törlése

Docker Compose szolgáltatások

db - PostgreSQL 17.6 adatbázis (5432-es port) backend - .NET backend API (8080-as port) typegen - Swagger típusgenerátor (egyszeri futás) web - Next.js webalkalmazás (3000-es port)

Indítás:

./script/docker-start.ps1 → script futtatása a **projekt mappából powershellben**

Build folyamat (web Dockerfile):

1. deps szakasz: pnpm install --shamefully-hoist
2. build szakasz: Next.js build Turbopack-kel
3. runner szakasz: Minimális production image

Turborepo konfiguráció

dev: cache kikapcsolva, persistent

build: dependsOn ["^build"], kimenet: .next/**, dist/**, build/** start: dependsOn ["build"], persistent test: cache kikapcsolva

Tesztelés

A frontend tesztek a következő fájlokban találhatók:

Zustand store tesztek:

- packages/hooks/src/useAuthStore.test.ts
- packages/hooks/src/useCalendarStore.test.ts
- packages/hooks/src/useContextMenuStore.test.ts
- packages/hooks/src/useProfileStore.test.ts

Utility tesztek:

- packages/utils/src/colors.test.ts
- packages/utils/src/calendar/calendarGridFactory.test.ts
- packages/utils/src/calendar/dateMethods.test.ts
- packages/utils/src/calendar/Month.test.ts
- packages/utils/src/calendar/Week.test.ts

Teszt futtatás: pnpm test

Platformfüggő különbségek

Jellemző	Web	Mobil *
Keretrendszer	Next.js	Expo (React Native)
Navigáció	Next.js App Router	Expo Router (Tabs)
Téma kezelés	NextThemeProvider	React Navigation Theme
Token tárolás	HTTP cookie-k	expo-secure-store
Auth header	Cookie (automatikus)	Bearer token (header)
Build rendszer	Turbopack	Metro bundler
Platform detektálás	isNative() = false	isNative() = true
UI framework	Tamagui (web)	Tamagui (native)
Deployment	Docker + Next.js	Expo build
Scrollbar	Egyedi CSS scrollbar	Natívscrollbar

Biztonsági megoldások

- JWT alapú hitelesítés (access + refresh token)
- Automatikus token frissítés 401-es válasz esetén
- Védett útvonalak (ProtectedRoute komponens)
- Kérések sorba állítása token frissítés közben
- Auth végpontok kizárása a refresh logikából
- Natív: SecureStore (titkosított tárolás)
- Web: HttpOnly cookie-k (XSS védelem)
- Zod séma validáció minden űrlapon (kliens-oldali)
- Erős jelszó követelmények regisztrációnál

BACKEND

1. Áttekintés

A Planova backend egy .NET 8 alapú RESTful Web API, amely a Planova naptár- és produktivitás-alkalmazás szerveroldali logikáját biztosítja.

Az API JWT-alapú hitelesítést, Entity Framework Core-on keresztüli adatbázis-kezelést, valamint *SignalR valós idejű kommunikációt használ.

Keretrendszer: .NET 8 (C#)
Adatbázis: PostgreSQL (Npgsql provider)

ORM: Entity Framework Core 8

Hitelesítés: JWT Bearer (access + refresh token)

***Valós idejű**: Entity Framework Core SignalR

API dokumentáció: Swagger / OpenAPI (Swashbuckle)

Projektkezelés: User Secrets a szenzitív konfigurációhoz

2. Projektstruktúra

apps/backend/backendAPI/
Program.cs – Alkalmazás belépési pont és szolgáltatás-konfiguráció
appsettings.json – Alkalmazás beállítások és connection string

Controllers/ – API kontrollerek (9 db)

Models/ – Entitás modellek (9 db)

Models/Enums/ – Felsorolás típusok (5 db)

DTOs/ – Adatátviteli objektumok (modulonként csoportosítva)

Services/ – Üzleti logika és validálás (10 szolgáltatás)

Data/ – DbContext és modell konfigurációk

Data/ModelConfigurations/ – Fluent API entitás konfigurációk (9 db)

Hubs/ – SignalR hub és felhasználó-azonosító provider

Common/ – Közös hibakonstansok

Extensions/ – Controller extension metódusok

Migrations/ – EF Core migrációk (5 db)

apps/backend/backendTests/ – Egységtesztek (xUnit)

3. Konfiguráció és indítás

3.1. Adatbázis kapcsolat

A connection string az appsettings.json fájlban található, a felhasználónév és jelszó helyőrzőket ({USERNAME}, {PASSWORD}) User Secrets-ből tölti be:

Host=localhost;Port=5432;Database=Vizsgaremek;

Username={USERNAME};Password={PASSWORD};Trust Server Certificate=true

3.2. JWT konfiguráció

- Issuer és Audience: appsettings.json-ban (AppSettings szekció)
- Titkos kulcs: User Secrets-ben (jwt-secret-key)
- Token érvényesség: rövid élettartamú access token + 7 napos refresh token
- Claims: UserId (NameIdentifier), Email, ProfileId, IssuedAt, JwtId

3.3. Szolgáltatás-regisztráció

Minden üzleti szolgáltatás Scoped életciklussal regisztrálódik a DI konténerbe. A SignalR felhasználó-provider Singleton-ként regisztrálódik.

3.4. Middleware pipeline

HTTPS átirányítás → CORS → Authentication → Authorization → SignalR hub mapping → Controller mapping

3.5. CORS

Engedélyezett origin: http://localhost:3000

Engedélyezett: minden header, minden metódus, hitelesítő adatok (credentials)

4. Adatmodell (entitások)

4.1. User

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Email (string, unique, required) – E-mail cím
- PasswordHash (string, required) – Hash-elt jelszó
- RefreshToken (string, nullable) – Aktuális refresh token
- RefreshTokenExpiryTime (DateTime?)– Refresh token lejárati ideje
- CreatedAt (DateTime) – Létrehozás dátuma (UTC)

Kapcsolat: egy User-hez egy Profile tartozik

4.2. Profile

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Username (string, 3-64 kar., unique) – Felhasználónév
- Avatar (string, required) – Avatar azonosító
- IsPrivate (bool) – Privát profil jelző
- FirstName (string, max 128, nullable)– Keresztnév
- LastName (string, max 128, nullable) – Vezetéknév
- BirthDate (DateOnly, nullable) – Születési dátum
- UserId (Guid, FK) – Felhasználó hivatkozás

Navigációs tulajdonságok:

- User, Habits, Calendars, SharedCalendars, CalendarEntries, EventContributors, FriendAsUser1, FriendAsUser2

4.3. Calendar

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Name (string, 3-32 kar., required) – Naptár neve
- Color (int, 1-24) – Szín azonosító
- ProfileId (Guid, FK) – Tulajdonos profil

Navigációs tulajdonságok: Profile, CalendarEntries, SharedCalendars

4.4. CalendarEntry

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- EntryCategory (EntryCategory, required) – Bejegyzés kategória
- Name (string, 3-32 kar., required) – Bejegyzés neve
- Description (string, max 1024, nullable)– Leírás
- StartDate (DateTime) – Kezdő dátum
- EndDate (DateTime) – Záró dátum
- Location (string, nullable) – Helyszín
- NotificationTime (DateTime, nullable) – Értesítés ideje
- Color (int, nullable) – Egyedi szín (ha null: naptár színe)
- IsCompleted (bool, nullable) – Teljesítettség (feladatoknál)
- IsAllDay (bool) – Egész napos jelző
- CalendarId (Guid, FK) – Naptár hivatkozás
- CreatedBy (Guid, FK) – Létrehozó profil

Navigációs tulajdonságok: Calendar, Profile

4.5. Habit

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Name (string, 3-64 kar., required) – Szokás neve
- Description (string, max 1024, nullable) – Leírás
- HabitCategory (HabitCategory, required) – Szokás kategória
- Unit (Unit, required) – Mértékegység
- Goal (float, nullable) – Célérték
- Color (int, 1-24) – Szín azonosító
- Days (int, nullable) – Aktív napok
- ProfileId (Guid, FK) – Tulajdonos profil

Navigációs tulajdonságok: Profile

4.6. HabitLog

Mezők:

- Id (Guid, PK) – Egyedi azonosító
- Date (DateTime, required) – Naplózás dátuma (UTC)
- Value (float, nullable) – Mért érték
- IsCompleted (bool, nullable) – Teljesítettség
- HabitId (Guid, FK) – Szokás hivatkozás

Navigációs tulajdonságok: Habit

4.7. Friend

Mezők:

- User1ProfileId (Guid, PK, FK) – Első felhasználó profil
- User2ProfileId (Guid, PK, FK) – Második felhasználó profil
- Status (Status, required) – Barátság állapota
- CreatedAt (DateTime) – Létrehozás dátuma
- UpdatedAt (DateTime, nullable) – Utolsó módosítás

Összetett elsődleges kulcs: (User1ProfileId, User2ProfileId)

Navigációs tulajdonságok: User1Profile, User2Profile

4.8. SharedCalendar

Mezők:

- ProfileId (Guid, PK, FK) – Felhasználó profil
- CalendarId (Guid, PK, FK) – Megosztott naptár
- Role (Role, required) – Jogosultsági szint

Összetett elsődleges kulcs: (ProfileId, CalendarId)

Navigációs tulajdonságok: Profile, Calendar

*4.9. EventContributor

Mezők:

- ProfileId (Guid, PK, FK) – Felhasználó profil
- CalendarEntryId (Guid, PK, FK) – Naptárbejegyzés
- Status (Status) – Résztvevő állapota

Összetett elsődleges kulcs: (ProfileId, CalendarEntryId)

Navigációs tulajdonságok: Profile, CalendarEntry

5. Felsorolás típusok (ENUMS)

5.1. EntryCategory

- None = 0
- Task = 1
- Event = 2
- BirthDay = 3
- Anniversary = 4

5.2. HabitCategory

- None = 0
- Exercise = 1
- Reading = 2
- Mindfulness = 3

5.3. Role

- Editor = 0
- Viewer = 1

5.4. Status

- Pending = 0
- Accepted = 1
- Dismissed = 2

5.5. Unit

- None = 0
- Distance = 1
- Quantity = 2
- Time = 3

6. API végpontok (kontrollerek)

Minden kontroller az api/ útvonal-prefix alatt érhető el.

Az **[Authorize]** jelöléssel ellátott végpontok JWT hitelesítést igényelnek.

6.1. AuthController (api/Auth)

POST api/Auth/sign-up
Leírás: Új felhasználó regisztrálása

Body: SignUpRequestDTO { Email, Password }

Válasz: 200 OK / 400 BadRequest (hibaüzenet)
Logika: Tranzakcióban létrehozza a User-t, Profile-t és egy alapértelmezett Calendar-t. A felhasználónév az e-mail @ előtti részéből generálódik (egyediség biztosított).

POST api/Auth/sign-in
Leírás: Bejelentkezés

Body: SignInRequestDTO { Email, Password }

Válasz: 200 OK + TokenResponseDTO / 400 BadRequest

Logika: Jelszó ellenőrzés PasswordHasher-rel. Sikeres bejelentkezés esetén JWT access token és refresh token generálása. A tokenek HttpOnly cookie-kban is beállítódnak (web kliens számára) és a válasz body-ban is visszatérnek (mobil kliens számára).

GET api/Auth/refresh
Leírás: Token frissítés

Forrás: Cookie-kból VAGY Authorization + X-Refresh-Token header-ből
Válasz: 200 OK + TokenResponseDTO / 401 Unauthorized

Logika: A lejárt access tokenből kinyeri a UserId-t (ValidateLifetime

= false), ellenőrzi a refresh token érvényességét, majd új token-párt generál.

GET api/Auth/sign-out
Leírás: Kijelentkezés

Válasz: 200 OK

Logika: Törli az accessToken és refreshToken cookie-kat.

GET api/Auth/verify [Authorize]
Leírás: JWT érvényesség ellenőrzése
Válasz: 200 OK / 401 Unauthorized

6.2. CalendarController (api/Calendar)

GET api/Calendar [Authorize]
Leírás: Felhasználó összes naptárának lekérése

Válasz: 200 OK + GetCalendarDTO[]

Logika: UNION lekérdezéssel összevonja a saját naptárakat és a megosztott naptárakat.

POST api/Calendar [Authorize]
Leírás: Új naptár létrehozása

Body: CreateCalendarDTO { Name, Color }

Válasz: 200 OK / 400 BadRequest

Logika: Naptárnév validálás (3-32 karakter), majd mentés.

PUT api/Calendar/{calendarId} [Authorize]
Leírás: Naptár módosítása

Body: UpdateCalendarDTO { Id, Name, Color }

Válasz: 200 OK / 400 BadRequest

Logika: Jogosultság-ellenőrzés (tulajdonos vagy Editor szerepkörű megosztott felhasználó), validálás, majd frissítés.

DELETE api/Calendar/{calendarId} [Authorize]
Leírás: Naptár törlése

Válasz: 204 NoContent / 400 BadRequest

Logika: Kaszkád törlés: először a naptárbejegyzések, majd a megosztások, végül maga a naptár.

6.3. CalendarEntryController (api/CalendarEntry)

GET api/CalendarEntry/{calendarId}?startDate&endDate [Authorize]
Leírás: Naptárbejegyzések lekérdezése egy adott naptárhoz

Query: startDate (opcionális), endDate (opcionális)
Válasz: 200 OK + GetCalendarEntryDTO[] / 403 Forbid

Logika: Hozzáférés-ellenőrzés, opcionális dátumszűrés. A bejegyzés színe: ha saját színe van, az; különben a naptár színe.

POST api/CalendarEntry/query [Authorize]
Leírás: Bejegyzések kötegelt lekérdezése több naptárból

Body: GetCalendarEntriesRequestDTO { StartDate?, EndDate?, Ids[] }
Válasz: 200 OK + GetCalendarEntryDTO[]
Logika: Hozzáférhető naptárak szűrése, majd bejegyzések lekérdezése az elérhető naptárakból.

POST api/CalendarEntry [Authorize]
Leírás: Új bejegyzés létrehozása

Body: CreateCalendarEntryDTO

Válasz: 200 OK / 400 BadRequest

Logika: Szerkesztési jogosultság ellenőrzése, naptár létezésének ellenőrzése, dátum validálás.

PUT api/CalendarEntry/{id} [Authorize]
Leírás: Bejegyzés módosítása

Body: UpdateCalendarEntryDTO

Válasz: 200 OK / 400 BadRequest

DELETE api/CalendarEntry/{id} [Authorize]
Leírás: Bejegyzés törlése

Válasz: 204 NoContent / 404 NotFound / 403 Forbid

6.4. FriendsController (api/Friends)

GET api/Friends?stat=Accepted|Pending|Dismissed [Authorize]
Leírás: Barátok listázása állapot szerint

Query: stat (alapértelmezett: "Accepted")

Válasz: 200 OK + GetFriendsDTO[]

Logika: UNION lekérdezés mindkét irányból (User1 és User2 oldalról).

POST api/Friends/friend-request [Authorize]
Leírás: Barátkérés küldése

Body: AddFriendDTO { ProfileId }

Válasz: 200 OK / 400 BadRequest

Logika: Ellenőrzi, hogy nem létezik-e már barátság, és nem saját maga-e a célszemély. Sikeres mentés után SignalR-en keresztül valós idejű értesítést küld a célfelhasználónak.

PUT api/Friends [Authorize]
Leírás: Barátság állapotának módosítása (elfogadás/elutasítás)

Body: UpdateFriendStatus { ProfileId, Status }
Válasz: 200 OK / 404 NotFound

DELETE api/Friends/{friendId} [Authorize]
Leírás: Barátság törlése

Válasz: 200 OK / 404 NotFound

*6.5. HabitController (api/Habit)

GET api/Habit [Authorize]

Leírás: Felhasználó összes szokásának lekérése
Válasz: 200 OK + GetHabitsDto[]

GET api/Habit/{habitId} [Authorize]
Leírás: Adott szokás lekérése

Válasz: 200 OK + GetHabitsDto

POST api/Habit [Authorize]
Leírás: Új szokás létrehozása

Body: CreateHabitDto { Name, Description?, Habitcategory, Unit,

Goal?, Days?, Color }

Válasz: 200 OK

PUT api/Habit/{id} [Authorize]
Leírás: Szokás módosítása

Body: UpdateHabitDto

Válasz: 200 OK / 404 NotFound

DELETE api/Habit/{habitId} [Authorize]
Leírás: Szokás törlése (kaszkád: szokás naplók is törlődnek)

Válasz: 204 NoContent / 404 NotFound

***6.6. ProfileController (api/Profile)**

GET api/Profile [Authorize]
Leírás: Bejelentkezett felhasználó profiljának lekérése
Válasz: 200 OK + GetProfileDTO

Logika: A UserId claim alapján kéri le a profilt.
Tartalmazza: id, username, avatar, isPrivate, firstName, lastName, birthDate, userCreatedAt, userEmail.

PUT api/Profile/{id} [Authorize]
Leírás: Profil módosítása

Body: UpdateProfileDTO { Id, Username, Avatar, IsPrivate,

FirstName?, LastName?, BirthDate?, Email }

Válasz: 204 NoContent / 400 BadRequest

Logika: Felhasználónév egyediség-ellenőrzés, profiladatok validálása (név hossz, születési dátum jövőbeli ellenőrzés).

***6.7. SharedCalendarController (api/SharedCalendar)**

POST api/SharedCalendar [Authorize]
Leírás: Naptár megosztása több felhasználóval

Body: CreateSharedCalendarDTO { Accounts[{ ProfileId, Role }],

CalendarId }

Válasz: 200 OK / 400 BadRequest / 404 NotFound

Logika: Naptár létezésének ellenőrzése, majd SharedCalendar rekordok tömeges létrehozása.

PUT api/SharedCalendar/{id} [Authorize]
Leírás: Megosztott naptár jogosultságának módosítása

Body: UpdateSharedCalendarRoleDto { Role }
Válasz: 200 OK / 404 NotFound

DELETE api/SharedCalendar/{id} [Authorize]
Leírás: Megosztás törlése

Válasz: 204 NoContent / 404 NotFound

*6.8. EventContributorController (api/EventContributor)

Jelenleg üres kontroller, implementálás tervezett.

*6.9. TestNotificationController (api/TestNotification)

POST api/TestNotification/send-friend-request/{targetProfileId}
Leírás: Teszt végpont SignalR értesítés szimulálásához

Válasz: 200 OK

Megjegyzés: Csak fejlesztési célra, nem igényel hitelesítést.

7. Adatátviteli objektumok (DTO-k)

7.1. Auth DTOs

SignUpRequestDTO

- Email (string)
- Password (string)

SignInRequestDTO

- Email (string)
- Password (string)

TokenResponseDTO

- AccessToken (string, required)
- RefreshToken (string, required)

7.2. Calendar DTOs

CreateCalendarDTO

- Name (string, required) - Color (int)

GetCalendarDTO

- Id (Guid)
- Name (string) - Color (int)

UpdateCalendarDTO

- Id (Guid)
- Name (string, required) - Color (int)

AccessibleCalendarDTO

- CalendarId (Guid) - Role (Role enum)

7.3. CalendarEntry DTOs

CreateCalendarEntryDTO

- EntryCategory (EntryCategory enum)
- Name (string, required)
- Description (string, nullable)
- StartDate (DateTime)
- EndDate (DateTime)
- Location (string, nullable)
- NotificationTime (DateTime, nullable)
- Color (int, nullable)
- IsAllDay (bool, nullable) - CalendarId (Guid)

GetCalendarEntryDTO

- A fenti mezők + Id, IsCompleted, CalendarName, CreatedBy, CreatedByName

UpdateCalendarEntryDTO

- A CreateCalendarEntryDTO mezői + Id, IsCompleted

GetCalendarEntriesRequestDTO

- StartDate (DateTime, nullable)
- EndDate (DateTime, nullable)
- Ids (List<string>)

***7.4. Friend DTOs**

AddFriendDT
O - ProfileId
(Guid)

GetFriendsDTO

- ProfileId (Guid)
- Username (string)
- Status (Status enum)
- CreatedAt (DateTime)
- UpdatedAt (DateTime, nullable)

UpdateFriendStatus

- ProfileId (Guid)
- Status (Status enum)

***7.5. Habit DTOs**

CreateHabitDto

- Name (string, required)
- Description (string, nullable)
- Habitcategory (HabitCategory enum)
- Unit (Unit enum)
- Goal (float, nullable)
- Days (int, nullable) - Color (int)

GetHabitsDto

- A fenti mezők + ProfileUsername, HabitCategory

UpdateHabitDto

- A fenti mezők + Id

***7.6. Profile DTOs**

CreateProfileDTO

- Username (string, required)
- Avatar (string, nullable)
- IsPrivate (bool)
- FirstName, LastName (string, nullable)
- BirthDate (DateOnly, nullable)
- UserId (Guid)

GetProfileDTO

- Id (Guid)
- Username, Avatar, IsPrivate
- FirstName, LastName, BirthDate
- UserCreatedAt (DateTime)
- UserEmail (string)

UpdateProfileDTO

- Id (Guid)
- Username, Avatar (string, required)
- IsPrivate (bool)
- FirstName, LastName, BirthDate
- Email (string, required)

***7.7. SharedCalendar DTOs**

CreateSharedCalendarDTO

- Accounts (List<SharedCalendarAccount>)
- ProfileId (string)
- Role (Role enum)
- CalendarId (Guid)

GetAllSharedCalendarDto

- ProfileUsername (string)
- CalendarName (string)
- Role (Role enum)

UpdateSharedCalendarRoleDto

- Role (Role enum)

7.8. Generikus válasz

ServiceResponse<T>

- Data (T, nullable) – Visszatérési adat
- Success (bool) – Sikeres-e a művelet
- Message (string?) – Hibaüzenet sikertelen esetben

8. Szolgáltatás réteg (services)

Minden szolgáltatás interfész + implementáció párosként van megvalósítva, és Scoped életciklussal regisztrálódik a DI konténerbe.

8.1. AuthService (IAuthService)

Felelősség: Felhasználó hitelesítése és token-kezelés

Metódusok:

- SignUpAsync(SignUpRequestDTO) → ServiceResponse<Guid>
  E-mail formátum és jelszó erősség validálás, duplikált e-mail ellenőrzés, jelszó hash-elés (PasswordHasher<User>), mentés.
- SignInAsync(SignInRequestDTO) → TokenResponseDTO?

  Felhasználó keresése e-mail alapján, jelszó verifikáció, JWT access token és refresh token generálás.
- RefreshTokensAsync(TokenResponseDTO) → TokenResponseDTO?

  Lejárt access tokenből UserId kinyerése, refresh token validálás az adatbázisból, új token-pár generálás.

Belső segédmethods:

- CreateToken() – JWT token generálás (claims: UserId, Email, ProfileId, IssuedAt, JwtId)
- GenerateRefreshToken() – 32 bájtos kriptográfiai random token
- GenerateAndSaveRefreshTokenAsync() – Refresh token mentés DB-be (7 napos lejárat)
- GetUserIdFromExpiredToken() – UserId kinyerése lejárt tokenből
- ValidateRefreshTokenAsync() – Refresh token és lejárat ellenőrzés
- IsValidEmail() – E-mail formátum validálás
- IsValidPassword() – Jelszó szabályok ellenőrzése (min. 8 karakter, kis- és nagybetű, szám, speciális karakter, max 128 karakter)

8.2. UserRegistration (IUserRegistration)
Felelősség: Felhasználó regisztrálása tranzakcióban

Metódusok:

- RegisterUserWithProfileAndCalendarAsync(SignUpRequestDTO) → ServiceResponse<bool> Adatbázis-tranzakcióban:
  1. User létrehozása (AuthService.SignUpAsync)
  2. Profile létrehozása (felhasználónév az e-mail @ előtti részéből)
  3. Alapértelmezett Calendar létrehozása Hiba esetén: rollback.
- GenerateRandomUniqueUsername(email) → string Egyedi felhasználónév generálás az e-mail címből.

8.3. CommonValidationService (ICommonValidationService)

Felelősség: Általános validálási műveletek

Metódusok:

- EntityExists<T>(Guid id) → ServiceResponse<T?> Generikus entitás-létezés ellenőrzés.
- ValidateEnum<TEnum>(string value) → ServiceResponse<TEnum> Enum érték validálás és parse-olás.
- ValidateText(string, minLength?, maxLength?) → ServiceResponse<bool> Szöveg hossz validálás (alapért. 3-32 karakter).

8.4. CalendarValidationService (ICalendarValidationService)

Felelősség: Naptár CRUD műveletek validálása

Metódusok:

- ValidateCalendarCreation() – Név validálás
- ValidateAndGetCalendarForUpdateAsync() – Létezés, jogosultság és név validálás
- ValidateAndGetCalendarForDeletionAsync() – Létezés és jogosultság
- ValidateCalendarEditingPermissionAsync() – Szerkesztési jogosultság (tulajdonos VAGY Editor szerepkörű megosztott felhasználó; Viewer szerepkör esetén elutasítás)
- HasCalendarAccessAsync() – Hozzáférés ellenőrzés (tulajdonos VAGY bármilyen megosztás)
- GetAccessibleCalendarsAsync() – Elérhető naptárak listája szerepkörrel

8.5. CalendarEntryValidationService (ICalendarEntryValidationService) Felelősség:

Naptárbejegyzés műveletek validálása Metódusok:

- ValidateCalendarEntryCreationAsync() – Szerkesztési jogosultság, naptár létezés, dátum validálás
- ValidateAndGetCalendarEntryForUpdateAsync() – Létezés és validálás

Validálási szabályok:

- Csak Task kategóriájú bejegyzés lehet „teljesített"
- A záró dátum nem lehet korábbi a kezdő dátumnál

***8.6. ProfileValidationService (IProfileValidationService)**

Felelősség: Profil műveletek validálása

Metódusok:

- ValidateProfileCreationAsync() – Felhasználónév, születési dátum, userId validálás

- ValidateProfileUpdateAsync() – Egyedi felhasználónév ellenőrzés, profil létezés, közös szabályok
- ValidateUniqueUsername() – Felhasználónév egyediség

Validálási szabályok:

- Felhasználónév: 3-64 karakter
- Születési dátum: nem lehet jövőbeli

***8.7. FriendValidationService (IFriendValidationService)**

Felelősség: Barátság műveletek validálása

Metódusok:

- ValidateFriendCreationAsync() – Duplikált barátság és önbarátság ellenőrzés
- ValidateAndGetFriendForUpdateAsync() – Létezés és validálás
- FriendExists() – Barátság keresés mindkét irányban

***8.8. HabitValidationService (IHabitValidationService)**

Felelősség: Szokás műveletek validálása

Metódusok

*ValidateHabitCreationAsync() – (még nem implementált) ValidateHabitUpdateAsync() – Létezés és név validálás

***8.9. SharedCalendarValidationService (ISharedCalendarValidationService)**

Felelősség: Megosztott naptár műveletek validálása Metódusok:

- SharedCalendarCreationAsync() – Duplikált megosztás ellenőrzés
- FindSharedCalendar() – Megosztott naptár keresés

9. Hitelesítés és jogosultságkezelés

9.1. Token kezelés

Access token (JWT):

- Claims: UserId, Email, ProfileId, IssuedAt, JwtId
- Aláírás: SymmetricSecurityKey (HMAC)
- Validálás: Issuer, Audience, Lifetime, IssuerSigningKey
- ClockSkew: Zero (pontos lejárat)

Refresh token:

- 32 bájtos RandomNumberGenerator → Base64 string
- Az adatbázisban (User tábla) tárolódik
- 7 napos lejárat
- Rotáció: minden frissítéskor új refresh token generálódik

9.2. Token megszerzés sorrendje

A backend az alábbi sorrendben keresi a tokent:

1. accessToken cookie (webes kliens)
2. SignalR query string access_token (/notifications útvonalra)
3. Authorization: Bearer header (Swagger/natív kliens)

9.3. Jogosultsági szintek naptáraknál

- Tulajdonos: minden művelet engedélyezett
- Editor (megosztott): olvasás + szerkesztés engedélyezett
- Viewer (megosztott): csak olvasás engedélyezett

9.4. Controller extension metódusok

- GetUserId() – UserId kinyerése a ClaimTypes.NameIdentifier claim-ből
- GetProfileId() – ProfileId kinyerése az egyedi "ProfileId" claim-ből

*10. Valós idejű kommunikáció (SignalR)

10.1. NotificationHub

- Útvonal: /notifications
- A hub osztály üres, a funkcionalitást az IHubContext<NotificationHub> biztosítja a kontrollerekben (server-to-client push).

10.2. CustomUserProvider

- Az IUserIdProvider implementációja, amely a SignalR kapcsolatokat a JWT token „ProfileId" claim-je alapján azonosítja. Ez teszi lehetővé, hogy a Clients.User(profileId) hívás a megfelelő felhasználónak küldje az üzenetet.

10.3. Jelenleg implementált értesítések

- Barátkérés értesítés (FriendsController):

- Esemény neve: "ReceiveNotification"
- Payload: { Type, Name, Message, SentAt }

11. Adatbázis konfiguráció

11.1. AppDbContext

- DbSet-ek: Users, Profiles, Calendars, SharedCalendars,
- CalendarEntries, Habits, HabitLogs, Friends, EventContributors
- A modell konfigurációk Fluent API-val és annotációkkal vannak definiálva, az Assembly.GetExecutingAssembly()-ból automatikusan betöltődnek (ApplyConfigurationsFromAssembly).

11.2. Modell konfigurációk (IEntityTypeConfiguration)

9 konfiguráció a Data/ModelConfigurations/ mappában:

- CalendarConfig
- CalendarEntryConfig
- EventContributorConfig
- FriendConfig
- HabitConfig
- HabitLogConfig
- ProfileConfig
- SharedCalendarConfig
- UserConfig

11.3. Migrációk

5 migráció:

- 2026.01.15 – Initial (kezdeti séma)
- 2026.01.23 – MinorFixes
- 2026.01.23 – SecondMinorFixes
- 2026.01.28 – ThirdMinorFixes
- 2026.01.29 – NullableCalendarEntryColor (bejegyzés szín nullable-re)

12. Hibakezelés

12.1. Közös hibaüzenetek (Common/)

AuthErrors:

- InvalidEmail – „Invalid email format."
- InvalidPassword – Részletes jelszó-szabályok leírása
- UserAlreadyExists – „User with this email already exists."
- InvalidCredentials – „Invalid email or password."

CommonErrors:

- InvalidRoute – „Route ID and body ID do not match"
- ImATeapot – „You are not allowed to contribute to this calendar"

12.2. ServiceResponse<T> minta

Minden szolgáltatás ServiceResponse<T>-t ad vissza:

- Success = true/false a művelet eredménye
- Message tartalmazza a hibaüzenetet
- Data tartalmazza a visszaadott adatot

A kontrollerek a ServiceResponse alapján döntik el a HTTP válaszkódot (200/201/204/400/401/403/404).

12.3. Konkurens módosítás kezelés

Az Update műveleteknél (Calendar, CalendarEntry, Profile, Habit, SharedCalendar) DbUpdateConcurrencyException elkapása történik. Ha az entitás törlődött a módosítás közben, 404 NotFound válasz.

13. NuGet csomagok

- **Microsoft.AspNetCore.Authentication.JwtBearer 8.0.22**
- **Microsoft.EntityFrameworkCore 8.0.22**
- **Microsoft.EntityFrameworkCore.Design 8.0.22**
- **Microsoft.EntityFrameworkCore.Tools 8.0.22**
- **Microsoft.EntityFrameworkCore.Relational 8.0.22**
- **Microsoft.AspNetCore.SignalR 1.2.9**
- **Npgsql.EntityFrameworkCore.PostgreSQL 8.0.11**
- **Swashbuckle.AspNetCore 6.6.2**
- **System.IdentityModel.Tokens.Jwt 8.15.0**
- **Microsoft.VisualStudio.Web.CodeGeneration.Design 8.0.22**

TESZT DOKUMENTÁCIÓ

I. BACKEND TESZTEK (C# / xUnit)

A backend tesztek a backendTests projektben találhatók.
Az adatbázis-műveletekhez InMemory adatbázist használnak,
így valós adatbázis-kapcsolat nélkül futtathatók.
Keretrendszer: xUnit / .NET 8 / Entity Framework Core InMemory

1. **BEJELENTKEZÉS TESZTEK**
   Fájl: Auth/SignInTests.cs
   Az AuthService bejelentkezés és token frissítés funkcióinak tesztelése.

1.1 SignIn_ShouldReturnNull_WhenUserDoesNotExist
Null-t ad vissza, ha a felhasználó nem létezik az adatbázisban.

1.2 SignIn_ShouldReturnNull_WhenPasswordIsIncorrect
Null-t ad vissza, ha a jelszó helytelen.

1.3 SignIn_ShouldReturnTokens_WhenCredentialsAreValid
Sikeres bejelentkezés esetén access és refresh tokeneket ad vissza. Ellenőrzi, hogy mindkét token nem null és nem üres.

1.4 SignIn_ShouldSaveRefreshToken_WhenLoginSucceeds
Sikeres bejelentkezés után a refresh token mentésre kerül az adatbázisba, a lejárati idő jövőbeli dátumra van állítva.

1.5 RefreshTokens_ShouldReturnNewTokens_WhenTokensAreValid
Érvényes tokenekkel történő frissítés esetén új access és refresh tokeneket ad vissza.

1.6 RefreshTokens_ShouldReturnNull_WhenRefreshTokenIsInvalid
Null-t ad vissza, ha a refresh token érvénytelen (manipulált).

1.7 RefreshTokens_ShouldReturnNull_WhenAccessTokenIsInvalid
Null-t ad vissza, ha az access token érvénytelen (nem valós JWT).

2. **REGISZTRÁCIÓ TESZTEK**
   Fájl: Auth/SignUpTests.cs
   Az AuthService regisztráció funkciójának tesztelése.

   SignUp_ShouldReturnError_WhenEmailIsInvalid Hibát ad vissza, ha az e-mail cím formátuma érvénytelen (pl. "invalidemail.com" @ jel nélkül).

2.1 SignUp_ShouldReturnError_WhenPasswordIsInvalid
(Theory – 6 eset) Hibát ad vissza érvénytelen jelszó esetén:
- Túl rövid jelszó
- Túl hosszú jelszó (140+ karakter)
- Csak nagybetűs jelszó

- Csak kisbetűs jelszó
- Speciális karakter nélküli jelszó
- Szám nélküli jelszó

2.3 SignUp_ShouldCreateUser_WhenDataIsValid
Érvényes adatokkal sikeres regisztráció. Ellenőrzi, hogy a felhasználó létrejön az adatbázisban helyes e-mail címmel és nem üres jelszó hash-sel.

2.4 SignUp_ShouldHashPassword_WhenUserIsCreated
A jelszó hash-elve kerül tárolásra – a PasswordHash nem egyezik meg az eredeti jelszóval.

2.5 SignUp_ShouldReturnError_WhenEmailAlreadyExists
Hibát ad vissza, ha az e-mail cím már foglalt.

3. TOKEN TESZTEK

Fájl: Auth/TokenTests.cs
A token frissítési mechanizmus részletes tesztelése.

RefreshTokens_ShouldReturnNewTokens_WhenTokensAreValid
Érvényes tokenek frissítése esetén új access és refresh tokeneket ad vissza. Az új tokenek különböznek a régiektől, és az új refresh token mentésre kerül az adatbázisba.

3.1 RefreshTokens_ShouldReturnNull_WhenRefreshTokenIsInvali
d Null-t ad vissza, ha a refresh token érvénytelen.

3.2 RefreshTokens_ShouldReturnNull_WhenAccessTokenIsInvalid
Null-t ad vissza, ha az access token érvénytelen.

3.3 RefreshTokens_ShouldReturnNull_WhenRefreshTokenIsExpir
ed Null-t ad vissza, ha a refresh token lejárt. Manuálisan hoz létre egy felhasználót tegnapi lejárati dátummal.

4. ÁLTALÁNOS VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl: Services/CommonValidationServiceTests.cs
A CommonValidationService általános validációs metódusainak tesztelése.

EntityExists

4.1 EntityExists_ShouldReturnSuccess_WhenEntityExists Sikert ad vissza, ha az entitás létezik az adatbázisban.

4.2 EntityExists_ShouldReturnFailure_WhenEntityDoesNotExist
Hibát ad vissza ("User not found"), ha az entitás nem található.

ValidateEnum

4.3 ValidateEnum_ShouldReturnSuccess_WhenValueIsValid Sikert ad vissza érvényes enum értékkel (pl. "Editor").

4.4 ValidateEnum_ShouldReturnSuccess_WhenValueIsCaseInsensitive Sikert ad vissza kis-nagybetű érzéketlenül (pl. "viewer" → Viewer).

4.5 ValidateEnum_ShouldReturnFailure_WhenValueIsInvalid Hibát ad vissza ("Invalid Role value"), ha az enum érték nem létezik.

ValidateText

4.6 ValidateText_ShouldReturnSuccess_WhenTextIsValid Sikert ad vissza érvényes szöveg esetén.

4.7 ValidateText_ShouldReturnFailure_WhenTextIsTooShort Hibát ad vissza, ha a szöveg túl rövid (2 karakter, minimum 3).

4.8 ValidateText_ShouldReturnFailure_WhenTextIsTooLong Hibát ad vissza, ha a szöveg túl hosszú (33 karakter, maximum 32).

4.9 ValidateText_ShouldReturnFailure_WhenTextIsEmpty Hibát ad vissza, ha a szöveg üres (csak szóközök).

4.10 ValidateText_ShouldRespectCustomMinAndMaxLength Egyedi minimum és maximum hossz megadása esetén is helyesen validál.

5. PROFIL VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl: Services/ProfileValidationServiceTests.cs
Profil létrehozás, módosítás és felhasználónév egyediség tesztelése.

ValidateProfileCreationAsync

5.1 ValidateProfileCreation_ShouldReturnSuccess_WhenDataIsValid Sikert ad vissza érvényes profil adatokkal.

5.2 ValidateProfileCreation_ShouldReturnFailure_WhenUserDoesNotExist Hibát ad vissza ("User not found"), ha a UserId-hez nincs felhasználó.

5.3 ValidateProfileCreation_ShouldReturnFailure_WhenUsernameIsTooShort Hibát ad vissza, ha a felhasználónév túl rövid (2 karakter).

5.4 ValidateProfileCreation_ShouldReturnFailure_WhenBirthDateIsInFuture Hibát ad vissza ("Are you from the future?"), ha a születési dátum a jövőben van.

ValidateProfileUpdateAsync

5.5 ValidateProfileUpdate_ShouldReturnSuccess_WhenDataIsVali d Sikert ad vissza érvényes módosítási adatokkal.

5.6 ValidateProfileUpdate_ShouldReturnFailure_WhenUsernameA lreadyExists Hibát ad vissza ("Username already exists"), ha a felhasználónév foglalt.

5.7 ValidateProfileUpdate_ShouldReturnFailure_WhenProfileDoes NotExist Hibát ad vissza ("Profile not found"), ha a profil nem létezik.

ValidateUniqueUsername

5.8 ValidateUniqueUsername_ShouldReturnTrue_WhenUsernameI sTaken True-t ad vissza, ha a felhasználónév már foglalt.

5.9 ValidateUniqueUsername_ShouldReturnFalse_WhenUsername IsAvailable False-t ad vissza, ha a felhasználónév elérhető.

5.10 ValidateUniqueUsername_ShouldExcludeOwnId_WhenIdProvi ded Saját profil Id megadásával kizárja a saját profilt az ellenőrzésből.

6. NAPTÁR VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl: Services/CalendarValidationServiceTests.cs
Naptár létrehozás, módosítás, törlés, jogosultság és hozzáférés tesztelése.

ValidateCalendarCreation

6.1 ValidateCalendarCreation_ShouldReturnSuccess_WhenNameIs Valid Sikert ad vissza érvényes naptárnévvel.

6.2 ValidateCalendarCreation_ShouldReturnFailure_WhenNameIs TooShort Hibát ad vissza, ha a naptárnév túl rövid.

6.3 ValidateCalendarCreation_WithDTO_ShouldReturnSuccess_W henValid Sikert ad vissza DTO-val történő létrehozás esetén.

ValidateAndGetCalendarForUpdateAsync

6.4 ValidateCalendarUpdate_ShouldReturnSuccess_WhenOwnerU pdates Sikert ad vissza, ha a tulajdonos végez módosítást.

6.5 ValidateCalendarUpdate_ShouldReturnFailure_WhenCalendar DoesNotExist Hibát ad vissza ("Calendar not found"), ha a naptár nem létezik.

ValidateCalendarEditingPermissionAsync

6.6 ValidatePermission_ShouldReturnSuccess_WhenUserIsOwner
Sikert ad vissza, ha a felhasználó a naptár tulajdonosa.

6.7 ValidatePermission_ShouldReturnFailure_WhenUserIsViewer
Hibát ad vissza, ha a felhasználó csak megtekintő jogosultsággal rendelkezik a megosztott naptárban.

HasCalendarAccessAsync

6.8 HasCalendarAccess_ShouldReturnTrue_WhenUserIsOwner
True-t ad vissza, ha a felhasználó a naptár tulajdonosa.

6.9 HasCalendarAccess_ShouldReturnFalse_WhenUserHasNoAcc
ess False-t ad vissza, ha a felhasználónak nincs hozzáférése.

ValidateAndGetCalendarForDeletionAsync

6.10 ValidateCalendarDeletion_ShouldReturnSuccess_WhenOwner
Deletes Sikert ad vissza, ha a tulajdonos törli a naptárat.

6.11 ValidateCalendarDeletion_ShouldReturnFailure_WhenCalenda
rNotFound Hibát ad vissza ("Calendar not found"), ha a naptár nem található.

GetAccessibleCalendarsAsync

6.12 GetAccessibleCalendars_ShouldReturnOwnAndShared
Visszaadja a saját és a megosztott naptárakat is (1 saját + 1 megosztott).

7. NAPTÁR BEJEGYZÉS VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl: Services/CalendarEntryValidationServiceTests.cs
Naptár bejegyzés létrehozás és módosítás validációjának tesztelése.

ValidateCalendarEntryCreationAsync

7.1 ValidateEntryCreation_ShouldReturnSuccess_WhenDataIsVali
d Sikert ad vissza érvényes bejegyzés adatokkal.

7.2 ValidateEntryCreation_ShouldReturnFailure_WhenCalendarDo
esNotExist Hibát ad vissza, ha a megadott naptár nem létezik.

7.3 ValidateEntryCreation_ShouldReturnFailure_WhenStartDateIs
AfterEndDate Hibát ad vissza ("End date cannot be sooner than start date"), ha a kezdő dátum a befejező dátum után van.

ValidateAndGetCalendarEntryForUpdateAsync

7.4 ValidateEntryUpdate_ShouldReturnSuccess_WhenDataIsValid Sikert ad vissza érvényes módosítási adatokkal.

7.5 ValidateEntryUpdate_ShouldReturnFailure_WhenEntryDoesNotExist Hibát ad vissza ("Entry not found"), ha a bejegyzés nem létezik.

Jogosultság ellenőrzés

7.6 ValidateEntryCreation_ShouldReturnFailure_WhenViewerTriesToCreate Hibát ad vissza, ha egy megtekintő jogosultságú felhasználó próbál bejegyzést létrehozni megosztott naptárba.

8. MEGOSZTOTT NAPTÁR VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl: Services/SharedCalendarValidationServiceTests.cs
Megosztott naptár létrehozás és keresés validációjának tesztelése.

SharedCalendarCreationAsync

8.1 SharedCalendarCreation_ShouldReturnSuccess_WhenNotAlreadyShared Sikert ad vissza, ha a naptár még nincs megosztva az adott profillal.

8.2 SharedCalendarCreation_ShouldReturnFailure_WhenAlreadyShared Hibát ad vissza, ha a naptár már meg van osztva az adott profillal.

FindSharedCalendar

8.3 FindSharedCalendar_ShouldReturnSuccess_WhenFound Sikert ad vissza és visszaadja a megosztott naptár adatait.

8.4 FindSharedCalendar_ShouldReturnFailure_WhenNotFound Hibát ad vissza ("Calendar not found"), ha nem található.

9. SZOKÁS (HABIT) VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK

Fájl:
Services/HabitValidationServiceTests.cs A
szokás módosítás validációjának tesztelése.

ValidateHabitUpdateAsync

9.1 ValidateHabitUpdate_ShouldReturnSuccess_WhenDataIsValid Sikert ad vissza érvényes szokás módosítási adatokkal.

9.2 ValidateHabitUpdate_ShouldReturnFailure_WhenHabitDoesNotExist Hibát ad vissza ("Habit not found"), ha a szokás nem létezik.

9.3 ValidateHabitUpdate_ShouldReturnFailure_WhenNameIsTooShort Hibát ad vissza, ha a szokás neve túl rövid (2 karakter).

10. **BARÁT VALIDÁCIÓS SZOLGÁLTATÁS TESZTEK**
Fájl: Services/FriendValidationServiceTests.cs
Barátkérelem létrehozás, keresés és módosítás validációjának tesztelése.

ValidateFriendCreationAsync

10.1 ValidateFriendCreation_ShouldReturnSuccess_WhenFriendDoesNotExist Sikert ad vissza, ha a barátkapcsolat még nem létezik.

10.2 ValidateFriendCreation_ShouldReturnFailure_WhenFriendAlreadyExists Hibát ad vissza ("Friend already exists"), ha a barátkapcsolat már létezik.

10.3 ValidateFriendCreation_ShouldReturnFailure_WhenSameUser Hibát ad vissza ("Users cannot be the same"), ha a felhasználó saját magát próbálja hozzáadni.

FriendExists

10.4 FriendExists_ShouldReturnSuccess_WhenFriendshipExists Sikert ad vissza és visszaadja a barátkapcsolat adatait.

10.5 FriendExists_ShouldReturnFailure_WhenNoFriendship Hibát ad vissza ("Friend not found"), ha nincs barátkapcsolat.

ValidateAndGetFriendForUpdateAsync

10.6 ValidateFriendUpdate_ShouldReturnSuccess_WhenFriendExists Sikert ad vissza, ha a barátkapcsolat létezik és módosítható.

10.7 ValidateFriendUpdate_ShouldReturnFailure_WhenFriendNotFound Hibát ad vissza ("Friend not found"), ha nem található.

10.8 ValidateFriendUpdate_ShouldReturnFailure_WhenSameUser Hibát ad vissza ("Users cannot be the same"), ha a felhasználó saját magának próbálja módosítani a barát státuszát.

II. FRONTEND TESZTEK (TypeScript / Jest)

A frontend tesztek a packages/hooks és packages/utils csomagokban találhatók. **Keretrendszer:** Jest / TypeScript / Zustand állapotkezelő

1. **AUTENTIKÁCIÓS STORE TESZTEK**
Fájl: hooks/src/useAuthStore.test.ts
A useAuthStore Zustand store tesztelése. A verify és refresh API hívások mockolva vannak.

Kezdeti állapot

1.1 isAuthorized alapértéke false.
1.2 isLoading alapértéke true.

setIsAuthorized

1.3 Az isAuthorized értékét true-ra állítja.
1.4 Az isAuthorized értékét vissza tudja állítani false-ra.

verifyAuth

1.5 Ha a verify() sikeres, isAuthorized true lesz, isLoading false-ra áll, és a refresh() nem hívódik meg.

1.6 Ha a verify() sikertelen, a refresh()-re esik vissza. Sikeres refresh esetén isAuthorized true lesz.

1.7 Ha mind a verify() mind a refresh() sikertelen, isAuthorized false marad és isLoading false-ra áll.

1.8 A verifyAuth futása közben isLoading true, befejezés után false.

2. **KONTEXTUS MENÜ STORE TESZTEK**
   Fájl: hooks/src/useContextMenuStore.test.ts
   A useContextMenuStore Zustand store tesztelése – a jobb klikkes kontextus menü állapotát kezeli.

Kezdeti állapot

2.1 display alapértéke "none".
2.2 contextMenuOpen alapértéke false.
2.3 position alapértéke {x: 0, y: 0}.

2.4 fieldType alapértéke null.
2.5 date alapértéke null.

showMenu / hideMenu

2.6 A menü megjelenítésekor display "block" és contextMenuOpen true.
2.7 A menü elrejtésekor display "none" és contextMenuOpen false.
2.8 A hideMenu hívása már elrejtett állapotban is biztonságos.

setPosition

2.9 A pozíciót helyesen állítja be ({x: 150, y: 300}).

setDisplay

2.10 A display értékét "block"-ra állítja.
2.11 A display értékét "none"-ra állítja.

setFieldType

2.12 A fieldType-ot "cell"-re állítja.
2.13 A fieldType-ot "schedule"-ra állítja.

2.14 A fieldType-ot null-ra állítja.

setDate

2.15 A dátumot egy Date objektumra állítja (év, hónap, nap ellenőrzése).
2.16 A dátumot null-ra állítja.
2.17 Új Date példányt hoz létre (nem referenciát) – immutabilitás biztosítása.

3. NAPTÁR STORE TESZTEK

Fájl: hooks/src/useCalendarStore.test.ts
A useCalendarStore Zustand store tesztelése – naptár navigációs állapot kezelése (kiválasztott dátum, nézettípus, hónap/hét/nap léptetés).

Kezdeti állapot

3.1 selectedDate az aktuális időhöz közeli (5 másodpercen belül).
3.2 viewType alapértéke "month".
3.3 checkedCalendarIds alapértéke üres tömb.

setSelectedDate

3.4 A kiválasztott dátumot helyesen állítja be.
3.5 Új Date példányt hoz létre az immutabilitás érdekében.

setViewType

3.6 A nézet típusát "week"-re állítja.
3.7 A nézet típusát "day"-re állítja.
3.8 A nézet típusát "multiweek"-re állítja.

setCheckedCalendarIds

3.9 A naptár azonosítókat helyesen állítja be.
3.10 Az előző azonosítókat felülírja az újakkal.

incMonth / decMonth

3.11 Egy hónappal előre lép (március → április).
3.12 Egy hónappal hátra lép (március → február). 3.13 Decemberről januárra lép és az évet is növeli.
3.14 Januárról decemberre lép és az évet is csökkenti.

incWeek / decWeek

3.15 7 nappal előre lép.
3.16 7 nappal hátra lép.
3.17 Hónap határ átlépése 7 napos léptetésnél (március → április).

incDay / decDay

3.18 1 nappal előre lép.
3.19 1 nappal hátra lép.
3.20 Hónap határ átlépése előre (március 31 → április 1). 3.21 Hónap határ átlépése hátra (április 1 → március 31).

incYear / decYear

3.22 A megadott számú évvel előre lép.
3.23 A megadott számú évvel hátra lép.

resetToToday

3.24 A kiválasztott dátumot visszaállítja a mai napra.

4. PROFIL STORE TESZTEK

Fájl: hooks/src/useProfileStore.test.ts

A useProfileStore Zustand store tesztelése – felhasználói profil beállítások kezelése. A localStorage mockolva van.

Kezdeti állapot

4.1 weekStartsOn alapértéke "monday".
4.2 hour12 (12 órás formátum) alapértéke false.
4.3 locale egy Intl.Locale példány.
4.4 locale nyelve megegyezik a böngésző nyelvével.

setLocale

4.5 A locale-t magyar nyelvre ("hu-HU") állítja.
4.6 A locale-t német nyelvre ("de-DE") változtatja.

setWeekStartsOn

4.7 A hét kezdőnapját vasárnapra állítja. 4.8 A hét kezdőnapját vissza tudja állítani hétfőre.
4.9 A weekStartsOn értéket elmenti a localStorage-ba.

setHour12

4.10 A 12 órás formátumot true-ra állítja.
4.11 A 12 órás formátumot false-ra állítja.
4.12 Az hour12 értéket elmenti a localStorage-ba.

5. SZÍN KONTRASZT TESZTEK

Fájl: utils/src/colors.test.ts

A getContrastFromHSLA segédfüggvény tesztelése – HSLA szín alapján eldönti, hogy fekete vagy fehér szöveg legyen rajta.

Feketét ad vissza világos színekre

5.1 Tiszta fehér hsl(0, 0%, 100%) → "black"
5.2 Fehér alfa értékkel hsla(0, 0%, 100%, 1) → "black"
5.3 Világos sárga hsl(60, 100%, 80%) → "black"
5.4 Világos szürke hsl(0, 0%, 80%) → "black" 5.5 Világos cián hsl(180, 80%, 75%) → "black"
5.6 Világos zöld hsl(120, 60%, 70%) → "black"

Fehéret ad vissza sötét színekre

5.7 Tiszta fekete hsl(0, 0%, 0%) → "white"
5.8 Fekete alfa értékkel hsla(0, 0%, 0%, 1) → "white"
5.9 Sötét kék hsl(240, 100%, 25%) → "white"
5.10 Sötét piros hsl(0, 100%, 25%) → "white" 5.11 Sötét zöld hsl(120, 100%, 15%) → "white"
5.12 Sötét lila hsl(270, 80%, 20%) → "white"

Szóközös HSL szintaxis

5.13 hsl(210 50% 50%) – szóközzel elválasztott szintaxist is kezel.
5.14 hsl(0 0% 100%) → "black"

Tizedes értékek

5.15 hsla(210.5, 50.5%, 50.5%, 0.8) – tizedes értékeket is feldolgoz.

Hibás bemenet kezelése

5.16 Üres string → "Invalid HSLA string format" hibát dob.
5.17 Véletlenszerű szöveg → hibát dob.
5.18 Hiányos HSL érték → hibát dob.

Határesetek

5.19 50%-os szürke hsl(0, 0%, 50%) → "white" (brightness = 128).
5.20 55%-os szürke hsl(0, 0%, 55%) → "black" (brightness > 128).

6. HÉT SEGÉDFÜGGVÉNY TESZTEK

Fájl: utils/src/calendar/Week.test.ts
A Week segédosztály tesztelése – a hétnapjainak címkéi különböző lokalizációkban és formátumokban.

getWeekLabel

6.1 weekdayIndex 0, long, en-US → "Sunday"
6.2 weekdayIndex 6, long, en-US → "Saturday"
6.3 weekdayIndex 0, short, en-US → "Sun"

6.4 weekdayIndex 3, short, en-US → "Wed" 6.5 weekdayIndex 0, narrow, en-US → "S" getWeekdayLabels

6.6 7 darab címkét ad vissza.
6.7 Vasárnapi kezdésnél "Sun"-nal kezdődik, "Sat"-tel végződik.

6.8 Hétfői kezdésnél "Mon"-nal kezdődik, "Sun"-nal végződik.
6.9 Hétfői kezdéssel a teljes sorrend: Monday → Sunday. 6.10 Vasárnapi kezdéssel a teljes sorrend: Sunday → Saturday.
6.11 Minden címke nem üres string.

getWeekdayLabel (alias)

6.12 Ugyanazt az eredményt adja, mint a getWeekLabel.

7. HÓNAP SEGÉDFÜGGVÉNY TESZTEK

Fájl: utils/src/calendar/Month.test.ts
A Month segédosztály tesztelése – hónapok címkéi különböző formátumokban.

getMonthLabel

7.1 monthIndex 0, long, en-US → "January"
7.2 monthIndex 11, long, en-US → "December"
7.3 monthIndex 0, short, en-US → "Jan"
7.4 monthIndex 5, short, en-US → "Jun"
7.5 monthIndex 0, narrow, en-US → "J"
7.6 monthIndex 0, 2-digit, en-US → "01" 7.7 monthIndex 11, numeric, en-US → "12" getMonthsLabels

7.8 12 darab címkét ad vissza. 7.9 Az első "January", az utolsó "December".
7.10 Rövid címkék sorrendje: Jan, Feb, Mar, ..., Dec.
7.11 Minden címke nem üres string.

getMonthLabels (alias)

7.12 Ugyanazt az eredményt adja, mint a getMonthLabel.

8. DÁTUM METÓDUS TESZTEK

Fájl: utils/src/calendar/dateMethods.test.ts
Dátumkezelő segédfüggvények tesztelése.

getFirstDayOfTheYear

8.1 2026. január 1. → csütörtök (4)
8.2 2024. január 1. → hétfő (1)

getFirstDayOfMonth

8.3 2026. március 1. → vasárnap (0) 8.4 2024. február 1. → csütörtök

(4) getOrdinalDate

8.5 2026. január 1. → 1 (az év első napja)
8.6 2026. december 31. → 365 (az év utolsó napja) 8.7

2024. december 31. → 366 (szökőév utolsó napja)

getWeekNumberISO

8.8 2026. január 1. → 1. hét
8.9 2026. december 31. → 53. hét
8.10 2026. január 4. (vasárnap) → 1. hét

9. NAPTÁR RÁCS GENERÁTOR TESZTEK

Fájl: utils/src/calendar/calendarGridFactory.test.ts
A naptár rács generátor függvények tesztelése – különböző naptár nézetekhez (hónap, többhetes, heti, napi) generálják a cellákat.

generateMonthCells

9.1 Hat hetet átfogó hónapnál 42 cellát generál.
9.2 Öt hetet átfogó hónapnál 35 cellát generál. 9.3 Az első cella az előző hónapból vagy az aktuális hónap elsejéről.
9.4 Minden cella érvényes dátummal rendelkezik.
9.5 Az inCurrentMonth flag csak az aktuális hónap napjainál true.
9.6 Cellák kronológikus sorrendben vannak.
9.7 Vasárnapi/hétfői kezdésnél az első cella a megfelelő napra esik.

generateMultiweekCells

9.8 Mindig pontosan 28 cellát generál (4 hét).
9.9 Hétfői kezdésnél az első cella hétfő. 9.10 Vasárnapi kezdésnél az első cella vasárnap.
9.11 Cellák kronológikus sorrendben vannak.

generateWeekCells

9.12 Mindig pontosan 7 cellát generál.
9.13 Hétfői kezdésnél az első cella hétfő.
9.14 Vasárnapi kezdésnél az első cella vasárnap.

9.15 Az utolsó cella a hétkezdetét megelőző nap.
9.16 Cellák pontosan 7 egymást követő napot fednek le (86400000 ms).

generateDayCell

9.17 Egyetlen cellát ad vissza a megadott dátummal, inCurrentMonth = true.

generateGrid

9.18 Havi nézet: 6 sor 6-hetes hónapnál.
9.19 Havi nézet: 5 sor 5-hetes hónapnál.
9.20 Többhetes nézet: 4 sor.
9.21 Heti nézet: 1 sor.
9.22 Heti sor pontosan 7 cellát tartalmaz.
9.23 Napi nézet: 1 sor.
9.24 Napi sor pontosan 1 cellát tartalmaz.
9.25 Sor kulcsok ISO hétszám formátumúak (W01, W02, stb.).
9.26 Havi sorok mindegyike pontosan 7 cellát tartalmaz.